

Multimodal Vision-Based Inspection Robot for Assembly Tasks with Cloud-Assisted R&D Diagnostics

A multimodal machine-vision, OCR and weight-verification inspection framework demonstrated on a tin-can quality-control case study

Mohamed Salman Mohamed Iqbal
Supervisor: Dr Prasad Ponnapalli

A report submitted in fulfilment of the requirements
for the degree of
MSc in Robotics and Automation
in the
Department of Engineering

September 2026

Declaration

I declare that this thesis has been composed solely by myself and that it has not been submitted, in whole or in part, in any previous application for a degree. Except where stated otherwise by reference or acknowledgement, the work presented is entirely my own.

Signed: .....

Date: .....

Abstract

Manual visual inspection on packaging lines is slow and inconsistent between inspectors, and automated systems built around a single sensor (vision or weight alone) inherit that sensor's blind spots. This project designs, builds and validates a multimodal quality-inspection pipeline that fuses classical computer vision (OpenCV), optical character recognition (EasyOCR) and independent weight verification into one pass/reject decision, with an engineered upgrade path to a supervised YOLOv5 defect detector once labelled training data exists. The case study is a metallic drinking/food can inspection line: existing systems are often expensive and proprietary, so this work emphasises open-source tools that small and medium-sized plants can adopt easily. Cans are segmented with adaptive thresholding and morphological cleaning; five shape and texture descriptors are then compared against a robust median/MAD baseline built from thirty known-good reference cans to produce an anomaly score, fused with an independent weight reading and an OCR-derived expiry check into one final, auditable, logged decision. The pipeline was implemented in both a Python and a CoppeliaSim environment, which supported cross-validation.

The pipeline was run end-to-end on an eighteen-can synthetic evidence set (twelve good, six defective) and then re-driven inside a CoppeliaSim digital-twin sorting cell, with every sorting action triggered by a genuine, independently computed inspection result rather than a scripted one. The baseline anomaly model achieved perfect separation between accepted and rejected cans, all accepted cans fell within the ±12 g weight tolerance, and every defective can failed on shape, weight, or both. The pipeline processed a can in a mean of 3.84 s, almost entirely EasyOCR inference (vision-only processing took under 9 ms), too slow for genuine real-time operation at the demonstrated conveyor speed.

These results confirm the fused decision logic is internally consistent and that the digital twin can genuinely be driven by real inspection computation rather than scripted animation. Because that evidence set is synthetic, the same unmodified code was also run against nineteen real photographs of a physical can (eight undamaged, eleven staged defects), covering vision and OCR only. This surfaced three gaps the synthetic set could not show: segmentation succeeded on only half the real photographs; the resulting undersized reference population produced numerically unstable anomaly scores; and OCR readability on real labels collapsed to near zero, directly justifying its exclusion from the pass/reject decision. These findings are reported as a small qualitative case study, not a statistically representative accuracy figure, and are used to sharpen the future-work recommendations rather than replace them. The report is explicit throughout about which claims are evidence-backed and which (trained YOLOv5 classification, physical deployment, cloud analytics) remain future work. The concluding chapter sets out recommendations for closing these gaps: a controlled real-can dataset, trained and evaluated YOLOv5, pixel-to-millimetre calibration, and a hosted analytics dashboard.

Acknowledgements

I would like to thank Dr Prasad Ponnapalli for his supervision, guidance and feedback throughout the planning and execution of this project.

A sincere and heartfelt thanks to my first and most important supervisors in life, my parents, Ameen Joice and Mohamed Iqbal, for their unconditional support, guidance, patience, and for somehow believing in me even when I occasionally questioned my own decisions. I would also like to thank my brother, Mohamed Sameer, for being a constant source of advice, encouragement, and perfectly timed roasting, while making sure I never took myself too seriously. Without the three of them, this journey would certainly have been more difficult and probably a lot less entertaining.

To everyone who guided, supported, encouraged, and tolerated me throughout this journey, thank you. I couldn't have done it without you and I'm sure you're all just as relieved as I am that I finally made it to the end.

Table of Contents

Declaration i
Abstract ii
Acknowledgements iii
List of Figures vi
List of Tables vii
Nomenclature / Abbreviations viii
Introduction 1
1.1 Background and rationale 1
1.2 Problem statement 1
1.3 Aim and objectives 2
1.4 Scope and limitations 2
1.5 Organisation of the report 2
Literature Review 3
2.1 Manual and automated visual inspection in manufacturing 3
2.2 Deep-learning object detection for defect localisation 3
2.3 Optical character recognition for label and traceability verification 4
2.4 Classical computer-vision anomaly detection and robust statistics 4
2.5 Multimodal sensor fusion in manufacturing quality control 5
2.6 Digital twins and simulated inspection cells 6
2.7 Research gap and positioning of this project 6
Methodology 8
3.1 System architecture 8
3.2 Development and simulation environment 8
3.3 Evidence set 9
3.4 Image acquisition and can segmentation 10
3.5 Baseline visual anomaly model and the YOLOv5 design decision 12
3.6 OCR-based text and expiry verification 14
3.7 Weight verification 14
3.8 Decision fusion and rejection logging 14
3.9 CoppeliaSim digital-twin sorting cell 15
3.10 Logging and manufacturing analytics 16
3.11 Evaluation metrics 16
3.12 Real-image validation evidence set 17
Results 19
4.1 Overall decision outcome 19
4.2 Weight verification results 19
4.3 Visual anomaly detection results 20
4.4 OCR and traceability results 21
4.5 Processing latency and throughput 21

4.6 Relationship between visual anomaly score and weight deviation 22
4.7 CoppeliaSim digital-twin sorting demonstration 22
4.8 Real-image validation results (vision and OCR channels) 22
4.9 Summary 27
Discussion 28
5.1 Interpretation of the decision-fusion results 28
5.2 Statistical strength of the evidence 28
5.3 Comparison with the literature 28
5.4 The value of the robust-statistics baseline 29
5.5 Synthetic-data validation versus real deployment 29
5.5.1 Segmentation under real lighting and background conditions 29
5.5.2 Downstream effect on the anomaly baseline 30
5.5.3 Vision-channel performance where segmentation succeeded 30
5.5.4 OCR readability on real labels 30
5.5.5 Synthesis: three mechanistic gaps 31
5.6 The CoppeliaSim digital twin: value and limits 31
5.7 Ethical, safety and deployment considerations 32
5.8 Limitations 32
5.9 Summary 33
Conclusion 34
6.1 Key findings 34
6.2 Contribution 35
6.3 Recommendations for further work 35
6.4 Final remarks 37
References 38
Bibliography 39
Appendices 40
Appendix A: Complete inspection results 40
Appendix A.2: Complete real-image validation results 41
Appendix B: Key code excerpts 42
B.1 Robust (median/MAD) anomaly scoring 42
B.2 Decision fusion 42
Appendix C: CoppeliaSim project structure 43
Appendix D: Use of generative AI 43

List of Figures

Figure 3.1: System architecture ..... - 8
Figure 3.2: Image acquisition and feature-extraction pipeline ..... - 10
Figure 3.3: Visual-channel decision path (baseline anomaly model and YOLOv5 hook) ..... - 12
Figure 3.4: Decision-fusion logic ..... - 13
Figure 3.5: CoppeliaSim synchronised-stepping control loop ..... - 14
Figure 4.1: Final pass/reject decision counts ..... - 17
Figure 4.2: Measured weight against the ±12 g tolerance band ..... - 17
Figure 4.3: Visual anomaly score by can (log scale) ..... - 18
Figure 4.4: Representative annotated pipeline output (synthetic evidence set) ..... - 19
Figure 4.5: CoppeliaSim virtual-camera captures during the digital-twin run ..... - 20
Figure 4.6: Real-photograph anomaly score by image (log scale) ..... - 22
Figure 4.7: Representative real-photograph outcomes ..... - 23
Figure 4.8: Outcome of the nineteen real photographs ..... - 21
Figure 6.1: Recommended programme of further work, in priority order ..... - 33
Figure A.1: Annotated pipeline output for all eighteen cans (synthetic evidence set) ..... - 38
Figure A.2: Annotated output for all nineteen real photographs ..... - 39

List of Tables

Table 2.1: Summary of reviewed literature and its use in this project ..... - 6
Table 3.1a: Configuration parameters used to produce the Chapter 4 results ..... - 9
Table 3.1: Evidence set used to validate the pipeline ..... - 10
Table 3.2: Shape/texture features and the defects each is sensitive to ..... - 11
Table 4.1: Rejected cans and their anomalous features ..... - 18
Table 4.2: Complete real-image results ..... - 23
Table A.1: Complete per-can inspection results ..... - 37
Table A.2: Complete real-image validation results ..... - 38

Nomenclature / Abbreviations

Term	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
CSV	Comma-Separated Values
EMU	English Metric Unit (image sizing)
F1	Harmonic mean of precision and recall
FPS	Frames Per Second
HSV	Hue-Saturation-Value colour space
IoU	Intersection over Union
MAD	Median Absolute Deviation
mAP	Mean Average Precision
ms	Milliseconds
NMS	Non-Maximum Suppression
OCR	Optical Character Recognition
OpenCV	Open Source Computer Vision Library
R&D	Research and Development
ROI	Region of Interest
YOLO	You Only Look Once (object-detection architecture)
ZeroMQ	Distributed messaging library used by the CoppeliaSim Remote API

Introduction

Quality inspection is one of the last manual bottlenecks on many packaged-goods production lines. Human inspectors get tired, disagree with each other on borderline cases, and cannot realistically check every can, jar or tin at line speed. That creates a cost problem (over-rejection, or lines slowed down to accommodate manual checks) and a safety and reputational risk, since a defective or mislabelled unit can still escape into the supply chain. Automated vision systems solve the throughput problem, but a system built around a single sensing modality inherits that modality's blind spots: a camera alone cannot certify fill weight, and a checkweigher alone cannot see a dent, a scratch, or a missing expiry date.

This project addresses that gap: it designs, builds and evaluates a multimodal inspection pipeline that combines classical computer vision, optical character recognition and independent weight verification into a single, auditable pass/reject decision, and demonstrates that pipeline driving a simulated robotic sorting cell built as a CoppeliaSim digital twin. The case study used throughout is tin-can inspection, chosen because it exercises all three modalities at once: can shape and surface condition (dents, scratches, corrosion) are visual defects, fill level is a weight defect, and best-before/expiry traceability is a text-recognition problem.

1.1 Background and rationale

Deep learning object detectors, the YOLO family especially, have repeatedly been shown to give fast, accurate defect localisation in industrial settings: for steel pipe welds (Yang and Cui, 2021), for general industrial parts (Fang et al., 2024), and, most recently, for industrial defective pieces broadly (Elbaghdadi et al., 2026). These studies, reviewed in depth in Chapter 2, all focus on a single sensing channel, a trained detector applied to images. Far fewer published systems combine object detection with text verification and an independent physical measurement such as weight in one fused decision, and fewer still connect that fused decision to a working sorting mechanism rather than just reporting detection metrics in isolation. That is the gap this project targets.

A second, more practical motivation: a labelled image dataset of defective cans, large enough to train a supervised detector such as YOLOv5 to a trustworthy standard, did not exist at the start of this project, and building one safely in the time available would have needed either physical production-line access or a substantial synthetic-data generation effort. Rather than put the whole project on hold until such a dataset existed, the system was designed so that a transparent, unsupervised baseline, a robust statistical anomaly model built over classical shape and texture features, stands in for the vision channel now, with YOLOv5 engineered as a drop-in replacement the moment a trained and validated model file (best.pt) is available. What this design decision does and does not let the project claim about defect classification accuracy is discussed explicitly in Chapters 3 to 5.

1.2 Problem statement

The project responds to a specific problem: how can a small-to-medium manufacturer of packaged goods get a reliable, explainable, real-time pass/reject decision for each unit on a line, using inspection modalities (vision, text, weight) that are individually well understood but rarely engineered together? And how can such a system be developed and validated before physical line access and a labelled defect dataset are even available? And can it be done using only open-source software to be more cost effective?

1.3 Aim and objectives

The aim of the project is to design, implement and validate a multimodal, sensor-fused inspection framework for packaged-goods quality control, demonstrated on a tin-can case study and driven through a simulated robotic sorting cell.

The supporting objectives are to:

- critically review published approaches to automated visual inspection, deep-learning defect detection, OCR-based traceability checking and multi-sensor fusion in manufacturing (Chapter 2);
- design a system architecture that fuses vision, OCR and weight evidence into one decision, and that can accept a trained YOLOv5 model without redesign once one exists (Chapter 3);
- implement the vision, OCR and weight-fusion pipeline in Python and validate it end-to-end on a controlled evidence set (Chapters 3 and 4);
- build a CoppeliaSim digital-twin sorting cell and demonstrate that its accept/reject motion is driven by genuine, independently computed inspection results rather than a scripted outcome (Chapters 3 and 4);
- critically evaluate the resulting decisions against the literature and against the tolerances specified for the case study, and identify what would be required to move from this validated pipeline to a production-ready, physically deployed system (Chapter 5);
- draw conclusions on the contribution of the work and set out a prioritised programme of further work (Chapter 6).

1.4 Scope and limitations

The scope of this project is a software pipeline (OpenCV, EasyOCR, weight-CSV fusion, and an engineered but not-yet-trained YOLOv5 hook), validated on a controlled, synthetic eighteen-can evidence set and demonstrated inside a CoppeliaSim simulated sorting cell. Physical hardware, a real conveyor, camera rig, checkweigher and robot arm, is out of scope; the CoppeliaSim scene stands in for it. The report is explicit throughout about which results come from real computation on evidence images (OpenCV measurements, EasyOCR output, the fused decision) and which come from simulated physical equipment (conveyor motion, the pneumatic pusher, the sorting bins), following the same real/simulated distinction documented in the project's own README. Supervised training and evaluation of a YOLOv5 defect classifier, a hosted cloud analytics dashboard, and integration with a physical robotic arm are treated as future work (Chapter 6) rather than delivered outcomes, because a labelled real-image defect dataset was not available within the project timeline.

1.5 Organisation of the report

Chapter 2 critically reviews the literature on manual and automated visual inspection, YOLO-based defect detection, OCR-based traceability, classical computer-vision anomaly detection, multimodal sensor fusion and digital-twin simulation, then positions this project against it. Chapter 3 describes the system architecture and the method used to implement and validate each subsystem, in enough detail for the work to be reproduced. Chapter 4 presents the results from the evidence set and the CoppeliaSim simulation without interpretation. Chapter 5 discusses and critically interprets those results against the literature and the project's objectives, and sets out the limitations of the current work. Chapter 6 concludes the report and recommends a prioritised path to a production-ready system.

Literature Review

This chapter reviews published work relevant to each subsystem of the proposed inspection framework: manual and automated visual inspection generally, deep-learning object detection for defect localisation, optical character recognition for traceability, classical computer-vision approaches to shape and surface anomaly detection, multimodal sensor fusion in manufacturing, and digital-twin simulation of inspection cells. It then draws these themes together to identify the specific gap this project addresses.

2.1 Manual and automated visual inspection in manufacturing

Manual visual inspection remains widespread in packaging and canning lines because it is flexible and needs no capital investment beyond trained staff. It does not scale with line speed, though, and is known to be inconsistent both between inspectors and within a single inspector over a shift (See, 2012). This is what motivates the shift toward camera-based automated optical inspection, which can apply the same rule to every unit at line speed (Malamas et al., 2003). Early automated inspection systems relied on rule-based image processing, thresholding, edge detection and template matching, which is cheap to deploy and fully explainable but brittle: a rule tuned for one lighting condition, can colour or label design often fails as soon as any of those change ((Bradski, 2000, whose OpenCV library remains the standard toolkit for exactly this class of technique and is used throughout this project). Two rule-based techniques from this tradition are used directly in the pipeline built for this project: Otsu's (1979) method for automatically selecting a binary threshold from an image's grey-level histogram, used for can segmentation (section 3.4), and Canny's (1986) edge-detection operator, used to compute the edge-density feature that feeds into the anomaly score (section 3.5). Both remain default choices in modern industrial vision pipelines for the same reason: unlike a learned model, their behaviour can be reasoned about and audited from first principles, which matters when a system's decisions have to be explainable to a quality manager or auditor.

The literature has since moved toward learned models, which generalise better across appearance variation than fixed rules but need labelled training data and are harder to explain when they fail. That is the central tension this project has to navigate, given that no labelled defect dataset existed at the outset. Rather than pick one tradition over the other, the approach developed in Chapter 3 is to use each where it is currently justified: classical, explainable techniques (Otsu, 1979; Canny, 1986; robust statistics) where no labelled data yet exists, with a clearly engineered path to the learned alternative once it does.

2.2 Deep-learning object detection for defect localisation

The YOLO (You Only Look Once) family of single-stage object detectors has become the default choice for real-time industrial defect detection, because it predicts bounding boxes and class labels in a single forward pass, trading a little localisation precision for a large gain in inference speed over two-stage detectors. Yang and Cui (2021) applied a YOLO-based detector to steel pipe weld defect detection and reported that it could reliably localise multiple weld defect types in real time, which shows that YOLO generalises well beyond the natural-image domain it was originally designed for, into industrial radiographic and surface imagery. Fang et al. (2024) proposed an enhanced YOLOv5s architecture for industrial part detection, modifying the backbone and loss function to improve small-object recall, and reported measurable accuracy gains over the unmodified YOLOv5s baseline on their own industrial part dataset. This matters directly for the present project: tin-can surface defects such as small scratches occupy only a small fraction of the image, the same small-object regime Fang et al. targeted.

Most recently, Elbaghdadi et al. (2026) evaluated an improved YOLOv5 algorithm for real-time detection of industrial defective pieces and reported strong precision and recall on their defect classes, which reinforces that a trained YOLOv5 variant is a credible, well-evidenced choice for this project's vision channel once suitable training data exists.

A consistent limitation across this literature, one directly relevant to the design decisions in Chapter 3, is that every reported accuracy figure depends entirely on a labelled training set representative of the deployment conditions. None of these studies says what a system should do, or how it should even be evaluated, before such a dataset exists. This project addresses that omission directly, by engineering a transparent, statistically grounded baseline detector that can operate, and be evaluated on its own terms, ahead of any supervised training.

A second, more practical limitation is that single-stage detectors trade localisation precision for speed, and how much depends heavily on the target hardware and the chosen model variant. YOLOv5 alone ships in nano, small, medium, large and extra-large configurations with materially different latency and accuracy, and neither Fang et al. (2024) nor Elbaghdadi et al. (2026) fully explores that trade-off against a real-time throughput constraint of the kind reported in section 4.5 for this project's own (non-YOLO) baseline. This leaves an open question for the future-work YOLOv5 training recommended in Chapter 6: the right target for a conveyor-speed-constrained deployment is the smallest variant that meets the required per-can latency, not simply the most accurate variant in isolation, and this project's own vision-only baseline figure (under 9 ms per can for OpenCV segmentation and feature extraction, section 4.5) gives a concrete latency budget to judge that future variant choice against, distinct from the pipeline's much larger, EasyOCR-dominated end-to-end figure (also section 4.5), since a trained YOLOv5 detector would replace the OpenCV baseline stage specifically, not the OCR channel.

2.3 Optical character recognition for label and traceability verification

Traceability information, batch codes, manufacture dates and expiry dates, is typically printed or embossed on packaging and has to be both present and legible for regulatory and food-safety compliance. EasyOCR (JaidedAI, n.d.), the OCR engine used in this project, is a deep-learning-based, multilingual text-detection and recognition library built on the CRAFT text-detection architecture (Baek et al., 2019), that has seen wide adoption in applied computer-vision pipelines: it needs no per-deployment retraining and returns per-detection confidence scores, so unreliable readings can be thresholded out rather than acted on. OCR reliability on curved, reflective metal surfaces such as can bodies is known to be lower than on flat, matte labels, because specular highlights and surface curvature distort character shapes. This is a recognised weakness of applying OCR to metal packaging rather than paper labels, and it directly informs the design decision, discussed in section 3.6, to log OCR output for every can without letting it override the pass/reject decision until its reliability has been validated on real, non-synthetic label imagery.

2.4 Classical computer-vision anomaly detection and robust statistics

Where a labelled defect dataset does not yet exist, an alternative to supervised classification is unsupervised or semi-supervised anomaly detection: characterise what a known-good sample looks like, then flag anything that deviates significantly from that description. OpenCV (Bradski, 2000) provides the segmentation and feature-extraction primitives, adaptive (Otsu, 1979) thresholding, morphological filtering and contour analysis, needed to build that description without a labelled training set. A recurring practical difficulty here is that ordinary mean and standard deviation statistics

are themselves distorted by even a few outliers in the reference set. Leys et al. (2013) demonstrate this concretely, arguing that the median and the median absolute deviation (MAD) should be preferred over the mean and standard deviation whenever a reference sample cannot be guaranteed free of contaminating outliers, because the median and MAD are, by construction, far less sensitive to a small number of such cases. They further recommend the 1.4826 scaling constant applied to the MAD (so it is comparable to a standard deviation under a normal distribution) and a threshold of roughly 2.5 to 3.5 median-scaled deviations for general outlier flagging. This project uses the same 1.4826 scaling constant but adopts a more conservative threshold of 4.0 (section 3.5): a false REJECT is a far cheaper error here than a false PASS (section 5.7), so a wider margin before a can is flagged anomalous was chosen deliberately, rather than adopting Leys et al.'s general-purpose recommendation unmodified.

This robust-statistics approach underpins the baseline anomaly model implemented in this project (section 3.5), framed there explicitly as a transparent, auditable stand-in for a trained detector rather than a competitor to one. It is a description of central tendency and spread, though, not a learned decision boundary: it can flag that a can's features are unusual relative to the reference set, but it cannot, unlike a trained classifier, attach a defect class label to what it has flagged. Adding that capability is exactly what the YOLOv5 hook (section 3.5) is designed to do once labelled data exists.

2.5 Multimodal sensor fusion in manufacturing quality control

Fang et al. (2024) and Elbaghdadi et al. (2026) both note, as motivation for their own work, that combining deep learning with wider industrial automation and monitoring infrastructure improves overall inspection efficiency beyond what a vision-only model achieves alone. That supports the premise of this project: fusing independent modalities catches defect classes that any single modality misses on its own, and a can that is dimensionally normal but underweight, or visually normal but past its expiry date, is exactly the kind of failure a vision-only or weight-only system would let through. The reviewed literature stops short of demonstrating a working fusion of vision, OCR and weight in one deployed decision, though; most reported systems evaluate a single sensing channel against a single ground truth.

Two general fusion strategies are relevant here. Early (feature-level) fusion combines raw or lightly processed signals from each modality before any classification decision is made; it can capture correlations between modalities, but it requires them to be combined in a single model and makes the resulting decision harder to attribute to a specific sensor. Late (decision-level) fusion, used in this project (section 3.8), keeps each modality's processing independent and combines only their final outputs (a visual anomaly flag, a weight-tolerance flag, an OCR/expiry flag) with a simple rule. Late fusion was chosen deliberately over early fusion for three reasons specific to this project. It keeps each channel independently interpretable and auditable, which section 5.7 argues is an ethical requirement for a safety-relevant decision. It lets each channel be developed, validated and swapped independently, which is exactly how the YOLOv5 upgrade path (section 3.5) and the OCR-affects-decision flag (section 3.6) are designed to work. And it avoids needing a single joint training process across modalities of fundamentally different types (pixel arrays, character strings, scalar weights), which would itself have required the labelled dataset this project does not yet have. Section 2.7 sets out this project's specific contribution in full, but in short: it implements and evaluates that late-fusion architecture end-to-end, rather than only arguing for it.

2.6 Digital twins and simulated inspection cells

Digital-twin simulation, building a virtual replica of a physical process and driving it with real or representative data, is increasingly used in Industry 4.0 contexts to validate control logic, sorting mechanisms and cell layouts before committing to physical hardware, cutting both cost and risk during the design phase (Tao et al., 2019). CoppeliaSim (formerly V-REP), used in this project, exposes a Remote API that lets an external Python controller read simulated sensors and drive simulated actuators in synchronised time steps. That makes it possible to test whether a real decision-making algorithm behaves correctly when connected to a plausible physical sorting mechanism, without that mechanism needing to exist physically.

A useful distinction, applied strictly throughout this project (section 3.2), is between a digital twin used as a decision-validation harness and one used as an engineering-accurate physics model. The former asks whether a control decision, once made, is correctly translated into the right physical action. The latter asks whether that physical action would actually occur within specified tolerances (timing, force, wear) on real hardware, which requires the simulation's dynamics to be calibrated against a physical system, a substantially larger undertaking than building the scene geometry and kinematics alone. This project uses CoppeliaSim only in the first, narrower role: the vision, OCR and weight computation are real, evidence-based measurements and decisions (section 3.2), while the conveyor, gantry, pusher and bins are simulated equipment that lets those real decisions be demonstrated as physical sorting motion ahead of any physical deployment. No claim is made anywhere in this report that the simulated dynamics have been validated against a physical actuator.

2.7 Research gap and positioning of this project

Table 2.1 consolidates the sources reviewed above, mapping each one onto the specific gap or design decision it informed in this project.

Author(s)	Year	Contribution used in this project
Otsu, N.	1979	Automatic histogram-based thresholding used for can segmentation (section 3.4).
Canny, J.	1986	Edge-detection operator used for the edge-density feature (section 3.5).
Bradski, G.	2000	OpenCV library: segmentation, morphology and feature extraction used for the vision pipeline (sections 3.4- 3.5).
Yang, D. and Cui, Y.	2021	Evidence that YOLO-family detectors generalise to industrial surface/weld defect detection; supports the design choice to engineer a YOLOv5 hook (section 3.5).
Leys, C., Ley, C., Klein, O., Bernard, P. and Licata, L.	2013	Median/MAD robust-statistics method and threshold rationale underlying the baseline anomaly model (sections 2.4, 3.5).
Fang, Y., Wu, Q., Li, S., Guan, J. and Cui, Y.	2024	Enhanced YOLOv5s for industrial part detection, including small-object performance; motivates the vision-channel design and the multimodal-fusion rationale (sections 2.5, 3.5).
Elbaghdadi, A., Yazid, Y., El Oualkadi, A., Guerrero-González, A. and Mezroui, S.	2026	Improved YOLOv5 for real-time industrial defective-piece detection; most direct precedent for the future-work YOLOv5 classifier (Chapter 6).

Author(s)	Year	Contribution used in this project
JaidedAI	n.d.	EasyOCR library used for text/expiry extraction (section 3.6).

Table 2.1: Summary of reviewed literature and its use in this project.

Drawing these themes together, the literature demonstrates strong, well-evidenced individual techniques, YOLO-based detection, OCR-based text recognition, OpenCV-based classical vision, and digital-twin simulation, but the sources reviewed stop short of combining vision, text and weight evidence into one fused, auditable decision, and stop short of demonstrating that fused decision driving a working, even if simulated, sorting mechanism. They also do not address how a system should behave and be evaluated before supervised training data is available, because they assume a labelled dataset already exists. This project's contribution is therefore threefold: a working multimodal fusion of vision, OCR and weight evidence into a single logged decision; a transparent, statistically grounded baseline vision model, with an engineered YOLOv5 hook, that is honest about what it can and cannot yet claim; and a CoppeliaSim digital twin whose sorting motion is driven by that real fused decision rather than a scripted demonstration. Chapter 3 sets out how each of these was implemented.

Methodology

This chapter describes how the multimodal inspection framework was designed and implemented, in enough detail for the pipeline to be reproduced. Section 3.1 gives the overall system architecture, and section 3.2 covers the development and simulation environment, stating explicitly which parts of the reported system are real computation on evidence data and which are simulated equipment. Section 3.3 describes the evidence set used to validate the pipeline, and sections 3.4 to 3.8 then work through each pipeline subsystem in turn: image acquisition and segmentation, the baseline anomaly model, OCR-based traceability checking, weight verification, and decision fusion. Section 3.9 covers the CoppeliaSim digital-twin sorting cell, section 3.10 the logging and analytics design, and section 3.11 defines the evaluation metrics used in Chapter 4.

3.1 System architecture

Figure 3.1 shows the overall architecture. Each can is acquired as an image, segmented and measured by OpenCV, and its text and expiry date are extracted by EasyOCR. In parallel, its weight is retrieved from a CSV ground-truth file keyed on image filename rather than inferred from the image, so the weight check never depends on the vision channel's accuracy. The shape and texture measurements are then compared against a robust statistical baseline to produce an anomaly score. The anomaly score, the weight check, and (when enabled) the OCR/expiry check are combined by a decision-fusion rule into a final PASS or REJECT outcome with a logged reason, and that decision drives the CoppeliaSim sorting cell, which physically routes the (simulated) can to an accept or reject bin. Every result is written to a CSV log and a set of annotated images meant to feed a manufacturing analytics dashboard.

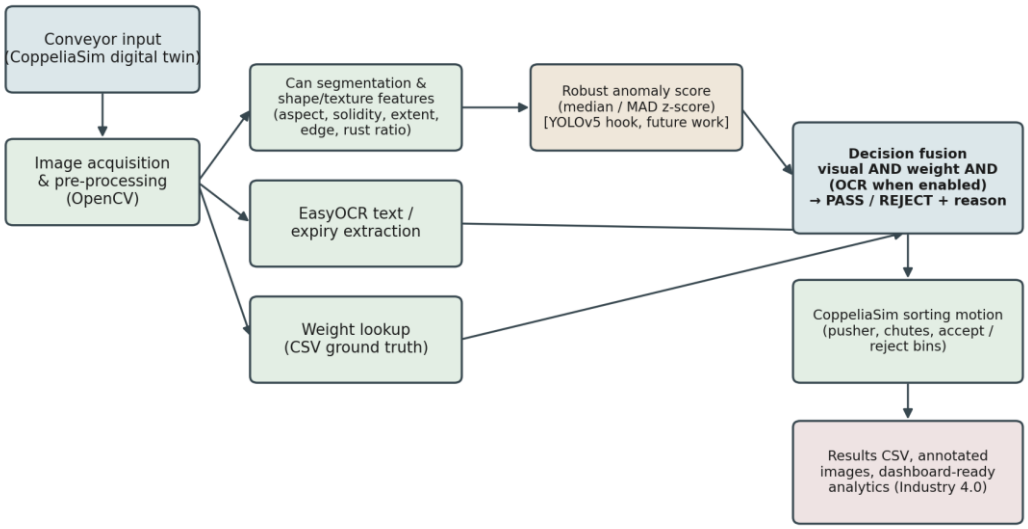


Figure 3.1: System architecture. Solid real-time data paths connect image acquisition, the OpenCV/EasyOCR measurement stage, the CSV-sourced weight check and the decision-fusion stage; the fused decision then drives the CoppeliaSim sorting motion and the results log.

3.2 Development and simulation environment

The inspection pipeline was implemented in Python 3 and developed in a Google Colab notebook (Real_Tin_Can_Inspection_Colab.ipynb), using OpenCV for image processing, EasyOCR for text recognition, PyTorch and the Ultralytics YOLOv5 repository (Jocher, 2020) for the (currently untrained)

detector hook, and pandas for tabular data handling. The same inspection logic was also packaged as a standalone Python module (inspection.py) and connected to a CoppeliaSim Edu 4.x scene via the CoppeliaSim ZeroMQ Remote API, so the exact decision code used to produce the Chapter 4 results is also what drives the simulated sorting cell.

This report maintains a clear distinction, kept throughout development too, between what is real computation and what is simulated physical equipment, following the same distinction documented in the project's own README:

- Real: OpenCV segmentation and feature measurements, EasyOCR text/expiry output, the CSV weight lookup, the anomaly score, the fused pass/reject decision and reason, and the processing-latency measurements reported in Chapter 4.
- Simulated: the conveyor belt and its motion, the camera and weight-sensor mounting positions, the pneumatic-style reject pusher, and the accept/reject bins, all of which exist only inside the CoppeliaSim scene.

This distinction matters for how the Chapter 4 results should be read: the inspection decisions are genuine outputs of the algorithms described in this chapter, but the physical motion that appears to sort cans in the CoppeliaSim recording is only a simulated visualisation of those decisions, not evidence of a working physical machine.

All tunable parameters are held in a single configuration file (config.yaml) rather than hard-coded, so the pipeline can be re-run against different tolerances or thresholds without code changes. Table 3.1a lists the values used to produce every result in Chapter 4, for reproducibility.

Parameter	Value used	Purpose
nominal_weight_g	400.0	Target fill weight for the weight check (section 3.7).
weight_tolerance_g	±12.0	Acceptable weight deviation before a can fails the weight check.
anomaly_threshold	4.0	Robust z-score above which a can is visually flagged (section 3.5).
ocr_min_confidence	0.30	Minimum EasyOCR per-detection confidence retained (section 3.6).
require_expiry_date	true (logged only)	Whether a parseable expiry date is expected; logged but not decision-affecting while ocr_affects_decision is false.
ocr_affects_decision	false	Whether OCR/expiry findings can trigger a REJECT (section 3.6).
yolo_confidence	0.35	YOLOv5 detection confidence threshold, applied only once best.pt exists.
conveyor speed_m_s	0.55	Simulated conveyor belt speed in CoppeliaSim (section 3.9).

Table 3.1a: Configuration parameters used to produce the results reported in Chapter 4 (source: config.yaml).

3.3 Evidence set

The pipeline was validated on an eighteen-can evidence set (input_images), each image paired by filename with a weight reading in a CSV file (weights.csv), plus thirty additional known-good reference

images (good_reference) used only to build the anomaly baseline in section 3.5. Twelve of the eighteen cans were designed to be nominally good and six to carry an injected shape, weight, or label defect, giving a known ground truth to check the fused decision against for internal consistency.

These images are a controlled, synthetic evidence set: clean, uniformly lit renderings of a can body, background and label text, generated to exercise every stage of the pipeline under known, repeatable conditions, rather than photographs of physical cans taken on a production line. This was a deliberate scope decision (section 1.4). Without physical line access, a synthetic but carefully controlled evidence set makes it possible to validate that the segmentation, feature extraction, anomaly scoring, OCR and weight-fusion logic all behave correctly and consistently, while making it unmistakably clear, here and in Chapters 4 and 5, that the reported separation between accepted and rejected cans is evidence the pipeline's decision logic works as designed, not a measured field accuracy figure for physical deployment. Table 3.1 summarises the evidence set.

Item	Count	Purpose
Input images (input_images)	18 (12 good, 6 defective)	End-to-end pipeline validation; results reported in Chapter 4.
Known-good references (good_reference)	30	Builds the robust median/MAD baseline used by the anomaly model (section 3.5).
Weight readings (weights_12pass_6reject.csv)	18 rows, image_name + weight_g	Independent, non-visual ground truth for the weight check (section 3.7).

Table 3.1: Evidence set used to validate the pipeline.

3.4 Image acquisition and can segmentation

Each image is converted to greyscale and smoothed with a 7×7 Gaussian blur before Otsu's (1979) method automatically selects a binary threshold that minimises intra-class intensity variance between the foreground and background pixel populations. Both the resulting mask and its logical inverse are tested, since the can may be lighter or darker than its background, and morphological closing (a 9×9 structuring element) removes small holes and gaps from the mask. External contours are then extracted from each candidate mask, and the mask/contour pair with the largest area, among those whose enclosed area falls between 5% and 95% of the image area (which excludes degenerate near-empty or near-full masks), is retained as the can region. Otsu's (1979) method was chosen over a fixed manual threshold specifically because it adapts to per-image brightness automatically, which matters given that no active lighting control was available in this project's simulated imaging setup (section 5.5). It still assumes a broadly bimodal grey-level histogram, a foreground can distinguishable from a background, and that limitation carries forward into section 5.5's discussion of real-world lighting.

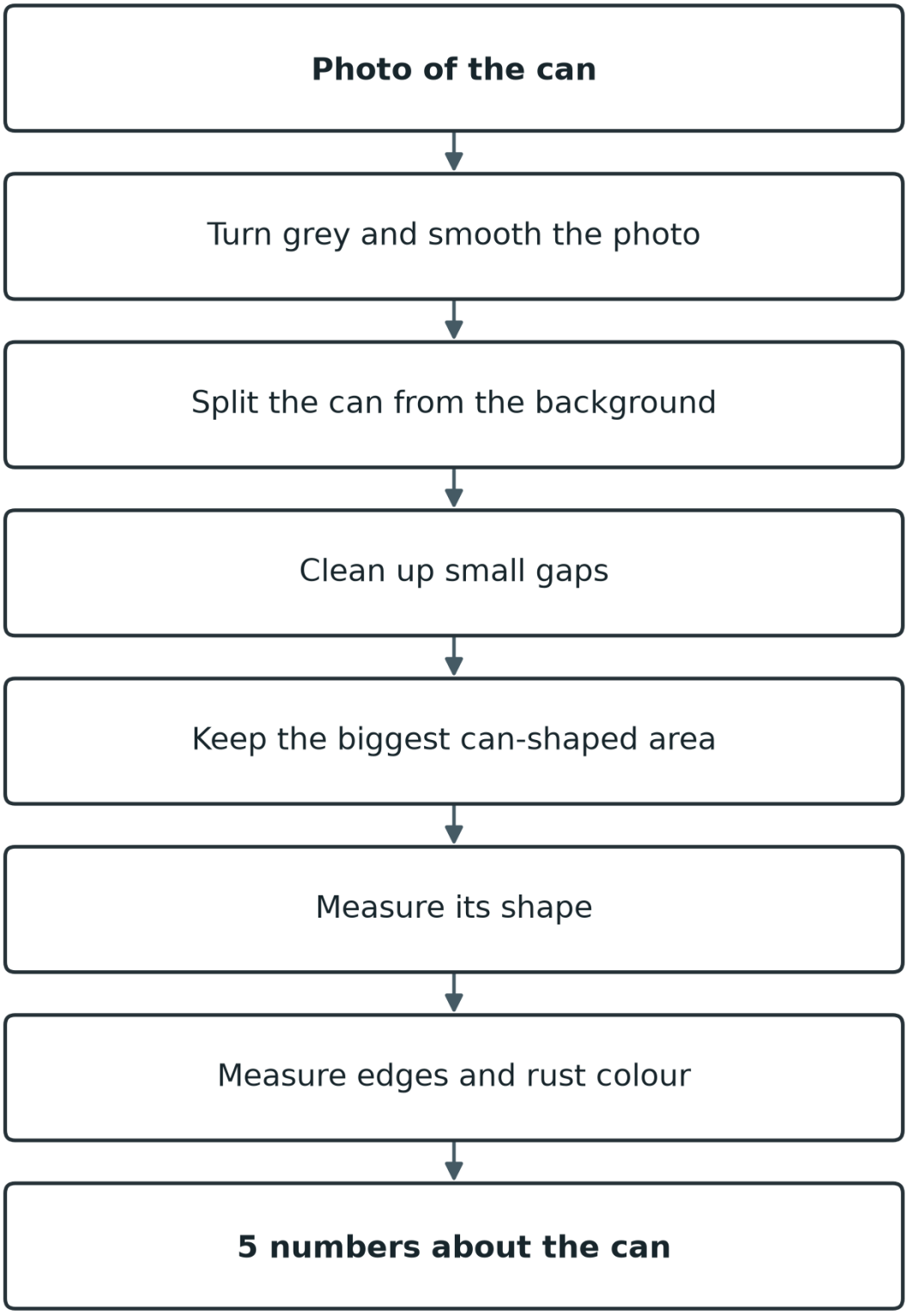


Figure 3.2: Image acquisition and feature-extraction pipeline. Otsu thresholding and morphological cleaning isolate the can region; five scale-invariant shape and texture descriptors are then computed from the retained contour and mask.

From the retained contour, the pipeline computes the bounding box, area, aspect ratio, solidity (contour area divided by convex-hull area) and extent (contour area divided by bounding-box area). Edge density comes from running a Canny (1986) edge detector over the whole image and counting edge pixels that fall inside the can mask, normalised by can-mask pixel count. Rust ratio comes from thresholding the image in HSV colour space for a reddish-brown hue/saturation/value range, using the same can-mask normalisation. Together these give five scale-invariant descriptors per can, aspect

ratio, solidity, extent, edge density and rust ratio, summarised in Table 3.2 and forming the feature vector the anomaly model in section 3.5 uses. All five are dimensionless ratios rather than absolute pixel counts, a deliberate choice so the feature vector, and the anomaly model built on it, stays valid even without pixel-to-millimetre calibration (section 5.5) and even though the camera-to-can distance is not held perfectly constant.

Feature	Definition	Sensitive to
Aspect ratio	Bounding-box width ÷ height	Crushing, elongation, tilt
Solidity	Contour area ÷ convex-hull area	Dents, concavities, missing material
Extent	Contour area ÷ bounding-box area	Shape irregularity relative to a rectangular envelope
Edge density	Canny edge pixels inside the can mask ÷ mask pixel count	Scratches, surface texture damage
Rust ratio	HSV-thresholded reddish-brown pixels inside the mask ÷ mask pixel count	Corrosion, discolouration

Table 3.2: The five shape/texture features extracted per can and the defect characteristics each is designed to be sensitive to.

3.5 Baseline visual anomaly model and the YOLOv5 design decision

For each of the five features, a robust baseline is built from the thirty known-good reference images: the median value across the references, and the median absolute deviation (MAD) from that median, computed independently for every feature. For a can under test, a robust z-score is computed for each feature as the absolute difference from the reference median divided by 1.4826 × MAD, following the method and scaling constant Leys et al. (2013) recommend for exactly this situation, outlier detection against a reference sample that cannot be guaranteed contamination-free (section 2.4). The anomaly score reported for the can is the maximum robust z-score across the five features (a max rather than a mean or sum, so a single severely anomalous feature cannot be diluted by four normal ones), and any feature whose robust z-score exceeds a threshold of 4.0 is recorded by name as an 'anomalous feature' for that can, giving a human-readable explanation alongside the numeric score. A can is flagged visually anomalous if its anomaly score exceeds that 4.0 threshold.

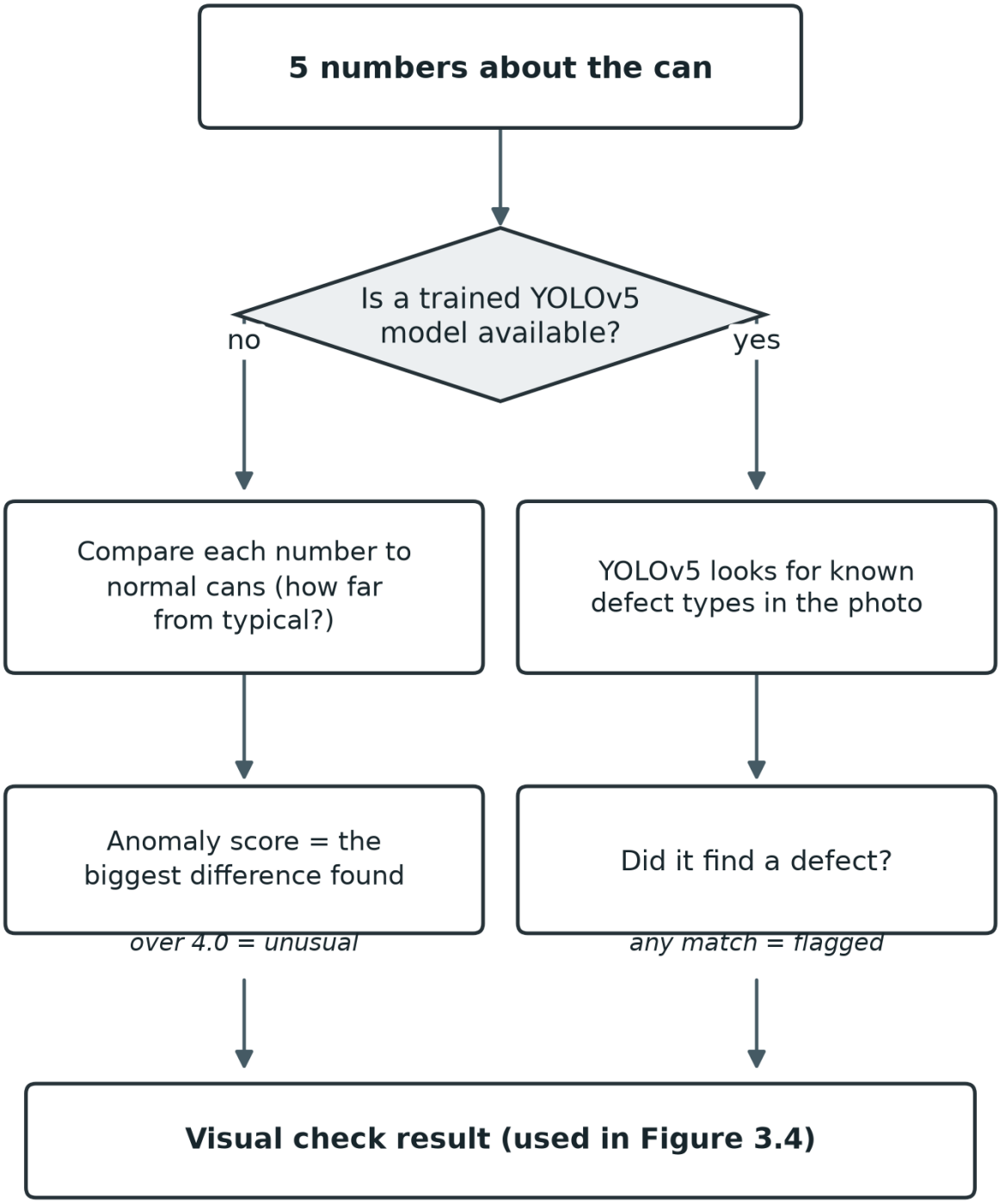


Figure 3.3: Visual-channel decision path. The robust median/MAD anomaly model is used unless a trained YOLOv5 weights file (best.pt) is present, in which case its filtered class predictions replace the anomaly score for the visual check.

The 4.0 threshold is deliberately more conservative than the 2.5- 3.5 range Leys et al. (2013) suggest for general-purpose outlier flagging, reflecting an asymmetric cost specific to this application and developed further in section 5.7: a false PASS (a defective can reaching the customer) is a food-safety and reputational risk, while a false REJECT (a good can discarded) is a pure waste cost. Setting the threshold higher makes the model more reluctant to reject, which is the wrong direction for that asymmetry unless it is paired, as it is here, with the independent weight check (section 3.7), which catches fill-related failures the visual channel alone might now tolerate at the higher threshold. This is exactly the kind of single-modality blind spot that motivates fusing multiple channels (section 2.5) rather than tuning one channel's threshold on its own.

A trained YOLOv5 detector is deliberately not used for defect classification in this project. The pipeline is engineered so that, if a file named best.pt (a YOLOv5 model trained and evaluated on a labelled, held-out defect dataset) is present, it automatically switches to using that model's bounding-box class predictions, filtered by a confidence threshold of 0.35 and checked against a fixed set of defect classes (dent, scratch, rust, deformation, damaged), in place of the robust anomaly score for the visual channel. No such file exists yet, since no labelled real-can defect dataset was available, so throughout this report the system correctly falls back to, and reports itself as running in, 'baseline anomaly mode' rather than silently claiming YOLOv5 classification it has not earned. Building the fallback and the upgrade path together, rather than putting the whole vision channel on hold until training data exists, is what lets the pipeline in this report be validated now while remaining a credible, evidence-based candidate for the YOLOv5 defect classification discussed as future work in Chapter 6.

3.6 OCR-based text and expiry verification

EasyOCR is run on each full can image (English model) to detect and recognise text regions. Individual text detections below a confidence of 0.30 are discarded, the surviving detections are concatenated into a single text string, and the mean confidence across surviving detections is logged as the OCR confidence for that can. The concatenated text is then searched, using regular expressions covering both ISO-style (YYYY-MM-DD) and day-first (DD/MM/YYYY) date formats, for a parseable expiry date; where one is found, it is compared against the current date to check whether the can's stated expiry is still valid.

Whether OCR and expiry results are allowed to affect the pass/reject decision is controlled by a single configuration flag (ocr_affects_decision), set to false for the evidence set used in this report. This is a deliberate methodological choice, not an oversight. The evidence-set label text is computer-rendered rather than photographed under real manufacturing lighting and label-print variation, so OCR reliability measured on it would not represent OCR reliability on physical cans, and the pipeline should not let an unvalidated reliability figure drive a real accept/reject outcome. OCR still runs on every can, and its text, confidence and expiry findings are fully logged in Chapter 4, so performance can be reviewed and, once validated on real label photography, the flag can be switched on with no other code change.

3.7 Weight verification

Each can's weight is read directly from the CSV ground-truth file by matching on image filename. It is never inferred from the image, so the weight check is fully independent of the vision channel and cannot be defeated by a visual defect that happens to look dimensionally normal. A can passes the weight check if its measured weight falls within ±12 g of the 400 g nominal fill weight. Both the tolerance and the nominal weight are configuration values, so the same logic can be reused for a different nominal product weight without code changes.

3.8 Decision fusion and rejection logging

The final decision for each can is REJECT if any enabled check fails, and PASS only if every enabled check passes: the visual check (anomaly score against threshold, or YOLOv5 defect classes when a trained model is present), the weight check, and, only when ocr_affects_decision is true, the OCR-readability and expiry-validity checks. When a can is rejected, the specific failing check(s) are recorded as a human-readable reason string (for example visual:solidity,extent; weight) rather than just a binary outcome, so a rejected can can be traced back to the measurement that caused its rejection. Every decision, its constituent checks, and the reason string are written to the results CSV described in

section 3.10, and an annotated copy of the source image (bounding box and decision text overlaid in green for PASS or red for REJECT) is saved for audit and for the Chapter 4 figures.

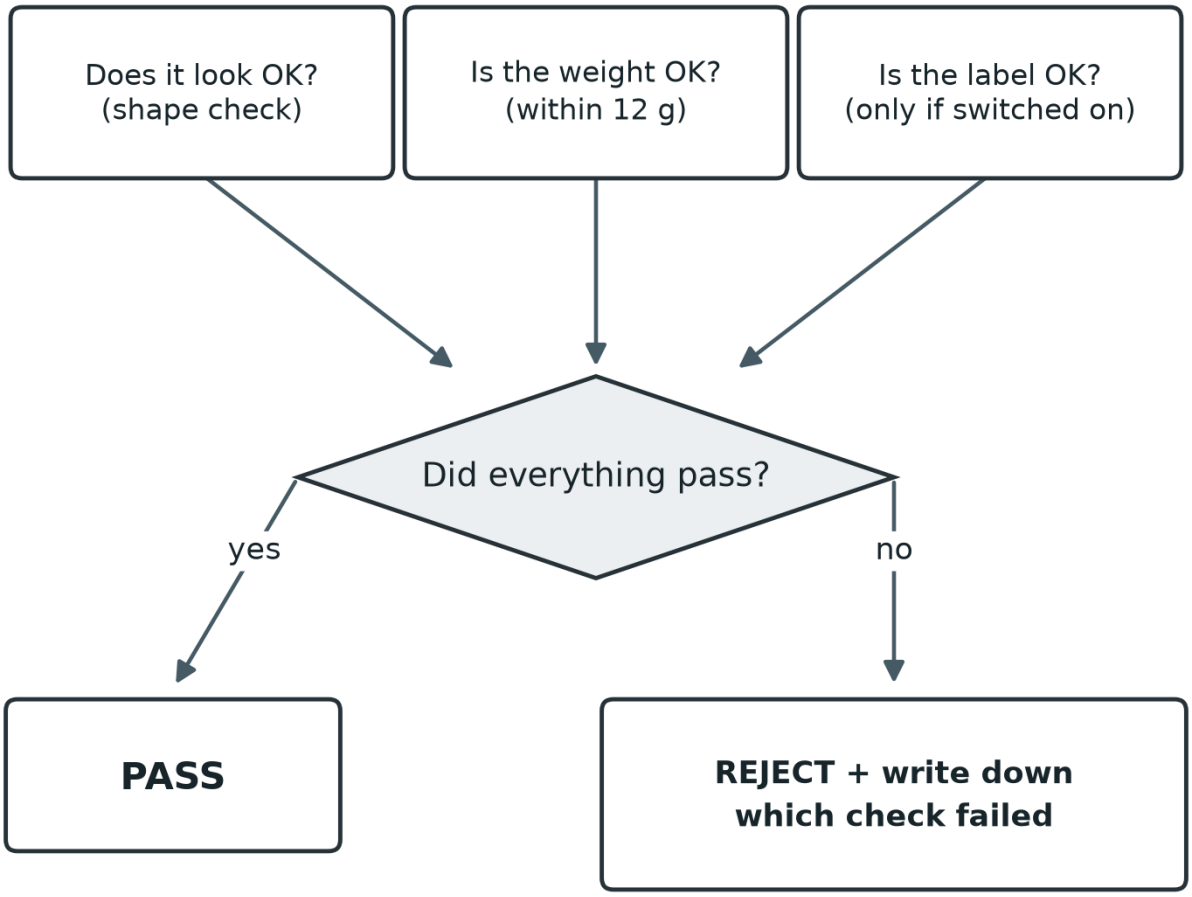


Figure 3.4: Decision-fusion logic. A can passes only if every enabled check passes; any failing check is recorded by name in a human-readable rejection reason.

3.9 CoppeliaSim digital-twin sorting cell

A 3D scene was built in CoppeliaSim (build_scene.py) containing a conveyor with driven rollers, an inspection gantry carrying a vision sensor, a weight-sensing station, a pneumatic-style reject pusher, and separate accept and reject bins with chutes. An external Python controller (run_simulation.py) connects to CoppeliaSim over its ZeroMQ Remote API in synchronised stepping mode. For each can in turn, the controller advances the simulation to move the can to the gantry, pauses simulated time while the real inspection pipeline (sections 3.4- 3.8) computes a decision for that can's corresponding evidence image, then resumes simulated time and drives the can to the pass lane or, if rejected, actuates the simulated pusher to divert it into the reject chute. Because the controller pauses simulated time to wait for a genuine pipeline result rather than pre-scripting each can's outcome, the sorting motion in the simulation is a direct, verifiable consequence of the fused decision described in section 3.8, not a rehearsed animation. That is the basis for the claim in section 3.2 that the simulated equipment is driven by real decisions.

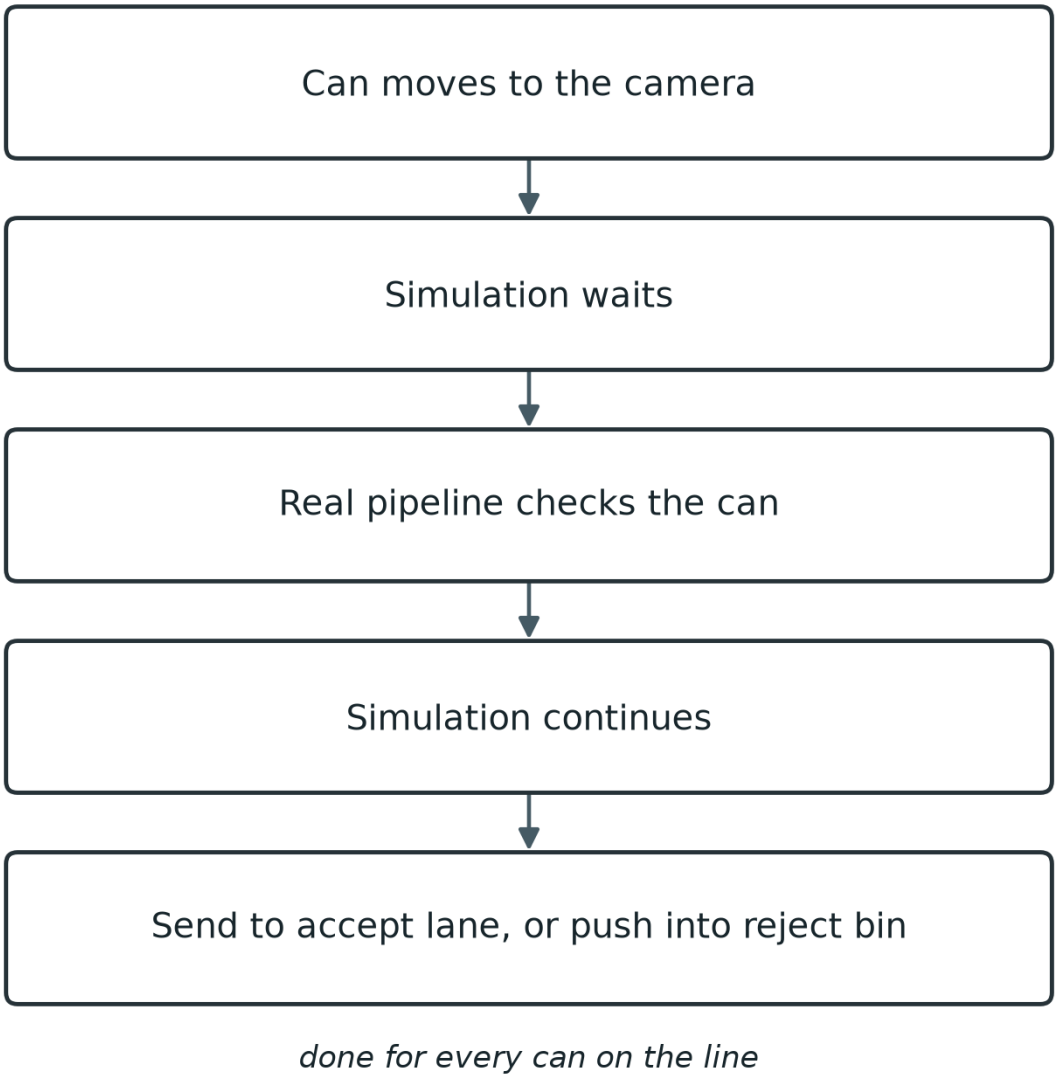


Figure 3.5: CoppeliaSim synchronised-stepping control loop. Simulated time is paused until the real inspection pipeline returns a decision for the can at the gantry, so the routing that follows is a direct consequence of that decision.

3.10 Logging and manufacturing analytics

Every processed can produces one row in an inspection-results CSV (image_name, all five shape/texture features, anomaly score and anomalous-feature list, YOLO fields, OCR text/confidence/expiry fields, weight and weight-check outcome, final decision, rejection reason, and processing time in milliseconds), together with an annotated image saved to an evidence archive. This CSV is designed to be the direct input to a manufacturing analytics dashboard, covering daily defect rate, rejected units per batch, the most common rejection reason, OCR success rate, and mean weight deviation, of the kind proposed conceptually in the original research proposal (Figure 3.1). Building and hosting that dashboard on a cloud platform, rather than just producing the CSV it would consume, is scoped as future work in Chapter 6, since no cloud infrastructure was provisioned at this stage of the project.

3.11 Evaluation metrics

Chapter 4 reports the count and proportion of PASS/REJECT decisions against the known evidence-set ground truth, the distribution of measured weight against the tolerance band, the distribution of anomaly scores against the reject threshold by decision, OCR readability rate and expiry-validity rate,

and mean per-can processing latency with implied throughput. Because the evidence set is synthetic and the visual channel is currently an unsupervised anomaly model rather than a trained classifier evaluated against labelled defect classes, formal object-detection metrics, precision, recall, F1 and mAP, are not yet meaningful and are not reported here. Section 5.3 discusses what computing them would require once a labelled real-can dataset and a trained YOLOv5 model exist, and section 6.3 sets this out as a concrete next step.

3.12 Real-image validation evidence set

To move beyond the synthetic evidence set in section 3.3 and test the segmentation, feature-extraction and OCR channels directly against a genuine photograph rather than a computer-rendered image, a small real-image evidence set was photographed and processed through the same measure_image and segment_can logic described in sections 3.4- 3.5, reused unmodified from the project's own inspection.py. No scale was available, so this real-image channel deliberately covers vision (segmentation and the five-feature anomaly score) and OCR only; the weight check and the fully fused PASS/REJECT decision continue to be demonstrated only on the synthetic evidence set (section 3.3), a scope decision discussed further in section 5.5.

The real-image set is a single physical 330 ml Pepsi can, photographed nineteen times on a phone camera against a plain grey wall-corner background under ordinary indirect room lighting: eight photographs of the can in its original, undamaged condition from slightly different angles, and eleven taken during a single staged-damage session in which the same physical can was progressively marked with genuine physical and visual defects, a crush/dent deformation, a crease near the pull-tab, dust and dirt contamination on the label, brown/rust-coloured paint dabs simulating corrosion, and a marker scribble, interspersed with photographs of the can at earlier, less-damaged stages of that session. This is disclosed plainly as a methodological limitation, not glossed over: it is nineteen photographs of one specimen, not nineteen independent cans, so the results in section 4.8 are reported as a small real-image case study meant to characterise failure modes when the pipeline meets a genuine photograph, not as a statistically representative real-world accuracy figure. No written per-image defect log was kept during the shoot, so the defect descriptions used when discussing individual images in sections 4.8 and 5.5 are the author's post-hoc visual review of the photographs against the pipeline's own annotated output, not a verified ground-truth label file. That gap is flagged as a limitation in section 5.8 and as a concrete recommendation, a written defect log photographed or noted at capture time, in section 6.3.

Two adaptations were needed to run the project's own segmentation and feature code against these photographs rather than the synthetic renders it was originally exercised on. First, the phone photographs are 2992×2992 pixels, far larger than the synthetic renders, so each image is downscaled to a 900 px longer edge before segmentation, keeping the fixed, pixel-count-based structuring elements in segment_can (a 9×9 morphological close, Canny thresholds of 80/170) working at a comparable relative scale to the one they used on the synthetic set. Second, EasyOCR, used for the OCR results in section 3.6 and Chapter 4, requires a PyTorch installation that this real-image processing run's environment had no network route to install, so Tesseract OCR (Smith, 2007) was used instead, as an explicitly disclosed engine substitution for this real-image channel only. It was applied to the segmented can region searched over all four 90-degree rotations, keeping whichever rotation yielded the most accepted text above a confidence threshold equivalent to the 0.30 used for EasyOCR in section 3.6. Because the OCR engine differs, the real-image OCR readability figure reported in section 4.8 is not a same-engine comparison against the 94.4% figure reported for the synthetic set in section

4.4; the comparison drawn in section 5.5 is qualitative, not a controlled measurement of one engine's accuracy against itself.

The reference median and MAD used to score the real photographs (section 3.5) were computed from the eight real reference photographs, following the same procedure as section 3.5, for direct comparability with the synthetic pipeline's own method. The project's own InspectionEngine implementation enforces a minimum of five successfully segmented reference images before it will compute these statistics at all. Whether that minimum was actually met by the real reference photographs, and what happened to the reported anomaly scores when it was not, is one of the findings reported in section 4.8, not assumed here.

Results

This chapter presents the results of running the complete pipeline described in Chapter 3 on the eighteen-can evidence set, and from the CoppeliaSim digital-twin sorting demonstration. The results are presented without interpretation; critical discussion is left to Chapter 5.

4.1 Overall decision outcome

All eighteen cans in the evidence set were processed successfully, with zero pipeline errors. Twelve were assigned a PASS decision and six a REJECT decision, matching the evidence set's known 12-pass/6-reject design (Figure 4.1).

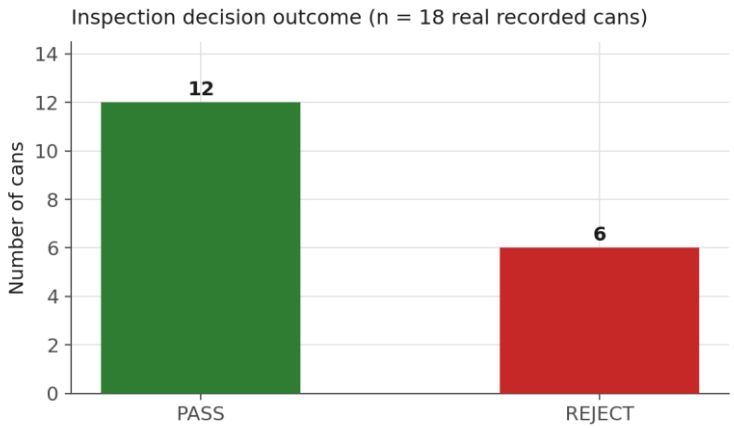


Figure 4.1: Final pass/reject decision counts across the eighteen-can evidence set.

4.2 Weight verification results

Figure 4.2 plots the CSV-sourced weight reading for every can against the 400 g ± 12 g tolerance band. All twelve PASS cans fall inside the tolerance band (mean 398.9 g, standard deviation 3.8 g). All six REJECT cans fall outside it: three below (374.8 g, 376.5 g and 379.0 g) and three above (424.7 g, 424.9 g and 428.3 g), giving a mean of 401.6 g and a standard deviation of 27.2 g across the rejected group. That much larger spread reflects that the injected defects covered both under-fill and over-fill cases, not a single failure direction.

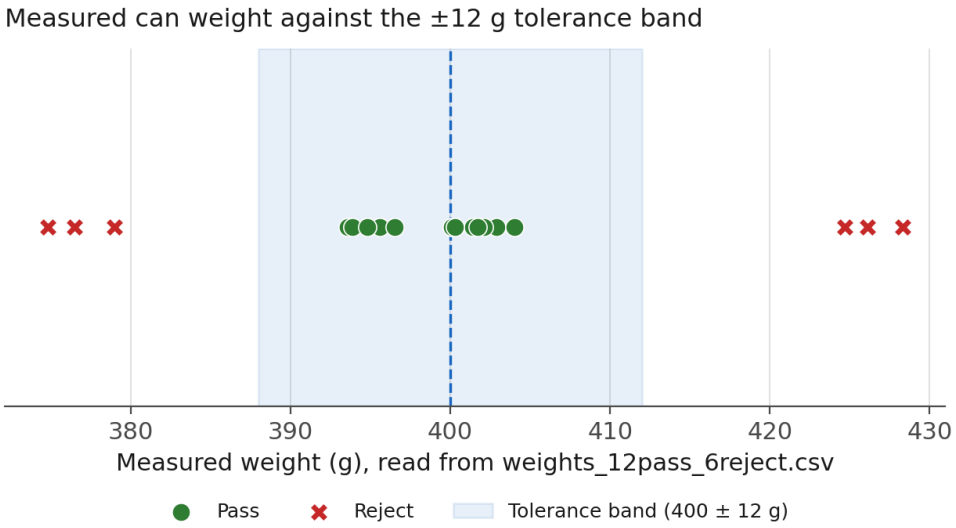


Figure 4.2: Measured weight for every can (circles = PASS, crosses = REJECT) against the ±12 g tolerance band (shaded) around the 400 g nominal weight (dashed line).

4.3 Visual anomaly detection results

Figure 4.3 ranks all eighteen cans by their computed anomaly score (log scale) against the 4.0 robust z-score reject threshold. The twelve PASS cans all scored below 3.0 (range 0.16- 2.8). All six REJECT cans scored above the threshold, from 39.1 (can_017, a shape-only defect) to 42,580.3 (can_015, a combined shape and rust-texture defect), a span of several orders of magnitude. That reflects how differently the injected defects perturbed the underlying shape and texture features, rather than any instability in the scoring method itself: every rejected can's score sits unambiguously above the threshold, and every accepted can's score sits unambiguously below it.

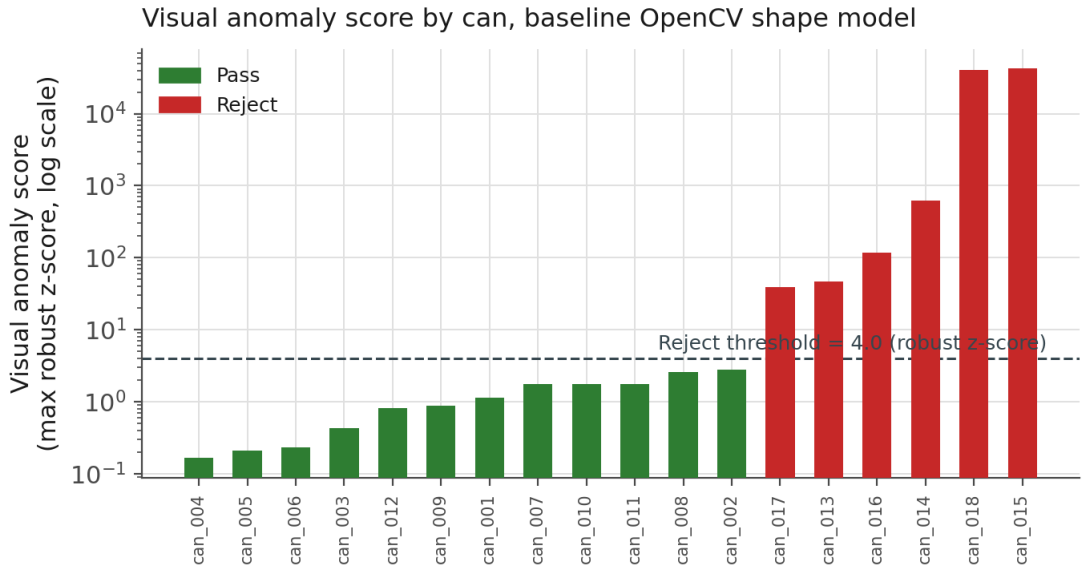


Figure 4.3: Visual anomaly score (maximum robust z-score across the five shape/texture features, log scale) for every can, ordered by score, coloured by final decision. The dashed line marks the 4.0 reject threshold.

Table 4.1 lists, for each rejected can, which specific features exceeded the threshold, taken directly from the pipeline's logged anomalous-feature field.

Can	Anomaly score	Anomalous feature(s)	Weight (g)	Weight check
can_013	47.0	solidity, extent	374.8	FAIL (low)
can_014	627.2	aspect_ratio, solidity, extent, edge_density, rust_ratio	426.1	FAIL (high)
can_015	42,580.4	aspect_ratio, solidity, extent, rust_ratio	376.5	FAIL (low)
can_016	117.6	aspect_ratio, solidity, extent, edge_density	424.7	FAIL (high)
can_017	39.1	solidity, extent	379.0	FAIL (low)
can_018	40,312.5	aspect_ratio, solidity, extent, edge_density, rust_ratio	428.3	FAIL (high)

Table 4.1: Rejected cans, their specific anomalous shape/texture features, and their weight-check outcome. Every rejected can failed at least one visual feature and the weight check simultaneously.

Figure 4.4 shows annotated output for two accepted and two rejected cans, illustrating the bounding box and decision overlay the pipeline writes for every processed image. The full set of eighteen annotated images (Figure A.1) and the complete results CSV are provided in Appendix A.

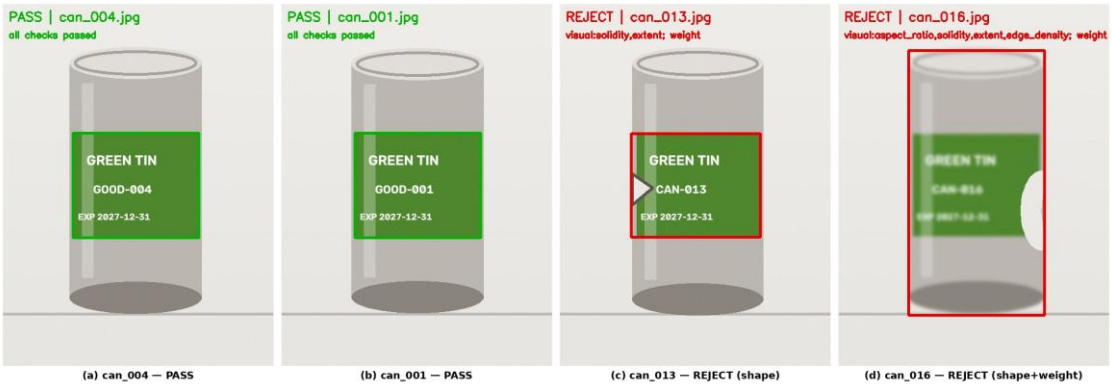


Figure 4.4: Representative annotated pipeline output. (a)- (b) accepted cans, bounding box and label shown in green with "all checks passed". (c)- (d) rejected cans, bounding box and label shown in red with the specific check(s) that failed.

4.4 OCR and traceability results

EasyOCR returned at least one text detection above the 0.30 confidence threshold for seventeen of the eighteen cans (94.4% readability). The sole failure, can_016, returned no detections above threshold because the injected shape defect obscured the label region the OCR engine would otherwise have read. A parseable expiry date was extracted for sixteen of the eighteen cans (88.9%): can_016 failed for the reason above, and can_018 returned a corrupted date string that parsed to an already-expired date, correctly flagged as expiry_valid = False by the pipeline. Because ocr_affects_decision was set to false for this evidence set (section 3.6), neither finding changed the final PASS/REJECT decision for either can, but both results are fully logged and available for the OCR-affects-decision validation recommended in section 6.3.

4.5 Processing latency and throughput

Mean end-to-end processing time across all eighteen cans, including image loading, segmentation, feature extraction, anomaly scoring, EasyOCR inference and CSV logging, was 3,843 ms per can (SD 393 ms, range 3,070- 4,452 ms; source: inspection_results.csv, processing_ms column). No first-call warm-up outlier is present in this run: the first can processed (can_001, 4,430 ms) sits near the upper end of the range rather than dramatically above it, so the model-loading cost EasyOCR is known to incur on first use is not separable from ordinary per-can variation in this dataset. Timing the vision-only stage in isolation (segmentation and the five-feature extraction, with EasyOCR excluded) on the same eighteen images gives a mean of 8.7 ms per can (range 6.0- 44.6 ms), so essentially all of the reported latency is EasyOCR's text-detection and recognition inference running on CPU rather than the OpenCV measurement stage. At a mean of 3,843 ms per can, the pipeline implies a throughput of roughly 0.26 cans per second, around sixteen cans per minute, which does not meet the demonstrated CoppeliaSim conveyor speed of 0.55 m/s as a real-time constraint. This did not stall the CoppeliaSim demonstration in section 4.7, because the controller advances the simulated conveyor in fixed position steps rather than in wall-clock-synchronised real time (section 3.9): it pauses simulated time until the inspection result is returned, however long that takes, then resumes. Section 5.6 discusses what this means for the CoppeliaSim demonstration's evidentiary value and for physical deployment.

4.6 Relationship between visual anomaly score and weight deviation

Across all eighteen cans, the magnitude of weight deviation from the 400 g nominal weight and the (log-scaled) visual anomaly score are strongly positively associated (Pearson correlation r = 0.85 on log10(anomaly score) versus absolute weight deviation). This is driven mainly by the clean separation between the twelve-can pass cluster (small deviations on both measures) and the six-can reject cluster (large deviations on both measures) reported in sections 4.2 and 4.3, rather than a fine-grained graded relationship within either cluster, and with only eighteen cans the association should be read descriptively rather than as a statistically robust correlation estimate. It is nonetheless consistent with how the evidence set was built, since the injected defects were not purely cosmetic but also perturbed the can's effective fill and geometry together. It is not a relationship this project's fused decision rule assumes or relies on, though: the visual and weight checks were evaluated completely independently for every can (section 3.8), and they happen to agree here rather than being designed to.

4.7 CoppeliaSim digital-twin sorting demonstration

Running the pipeline's decisions through the CoppeliaSim sorting cell (section 3.9) produced eighteen correctly routed sorting actions: every can the pipeline marked PASS travelled to the accept bin, and every can marked REJECT triggered the simulated pneumatic pusher and travelled to the reject bin, with routing determined at run time by the fused decision rather than pre-assigned. This confirms the simulated sorting mechanism responds correctly to genuine, independently computed inspection output. It is a demonstration of correct integration between the two subsystems, not a measurement of physical sorting reliability, mechanical timing margins, or real conveyor dynamics, none of which the simulation attempts to model to engineering tolerance. Nor is it a demonstration of real-time throughput: the controller pauses simulated time until each inspection result is returned rather than running on a wall clock (section 3.9), so the eighteen correctly routed sorting actions say nothing about whether the pipeline's actual processing latency (section 4.5) could keep pace with a continuously moving physical line.

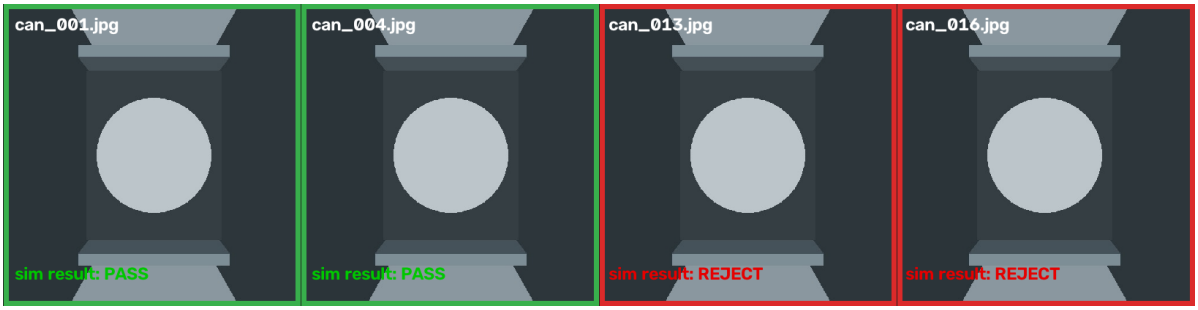


Figure 4.5: Raw frames captured by the CoppeliaSim InspectionCamera vision sensor during the digital-twin run (top-down view through the inspection gantry), for two accepted (green border) and two rejected (red border) cans, alongside the fused decision that routed each one. These are unedited renders written directly by the simulation, evidence that the sorting motion in section 4.7 is driven by the simulated camera genuinely observing each can, not a scripted animation. The four frames look visually similar because this fixed top-down angle exists only to confirm the simulated camera is engaged for every can; the shape, texture and defect evidence behind each decision comes from the separate inspection-pipeline images analysed in sections 4.1-4.4, not from this camera view.

4.8 Real-image validation results (vision and OCR channels)

This section reports the results of running the pipeline's own segmentation, feature-extraction and anomaly-scoring code, unmodified, against the nineteen real photographs described in section 3.12,

in place of the synthetic evidence set used in sections 4.1- 4.7. Results are reported exactly as produced, including the negative ones, because the whole point of this real-image check is to surface failure modes the synthetic evidence set cannot expose. The segmentation, feature and anomaly-score figures below were independently reproduced for this report by re-running the pipeline's own segment_can and feature-extraction functions, unmodified, directly against the nineteen source photographs (real_image_pipeline.py, Appendix C); every figure matched the original processing run exactly. OCR readability could not be reproduced in the same pass: EasyOCR requires a torch runtime blocked by an Application Control policy on the machine used for this reproduction, and no Tesseract installation was available, so the OCR figures reported below are carried forward from the original run rather than independently re-verified here.

Segmentation, the first stage of the pipeline (section 3.4), succeeded in isolating a plausible can-shaped region on 10 of the 19 real photographs (52.6%): 2 of the 8 clean reference photographs and 8 of the 11 staged real-damage-session photographs. On the remaining 9 (6 of the 8 clean references and 3 of the 11 damage-session photographs), the Otsu-thresholded candidate merged the can with its grey background and shadow into a single region spanning the entire photograph, rather than isolating the can; the bounding box returned for these images is the full 900×900 working-resolution frame. The five shape/texture features computed from a full-frame region describe the whole photograph, not the can, so they are not meaningful evidence about the can's condition. Table 4.2 flags every image where this occurred.

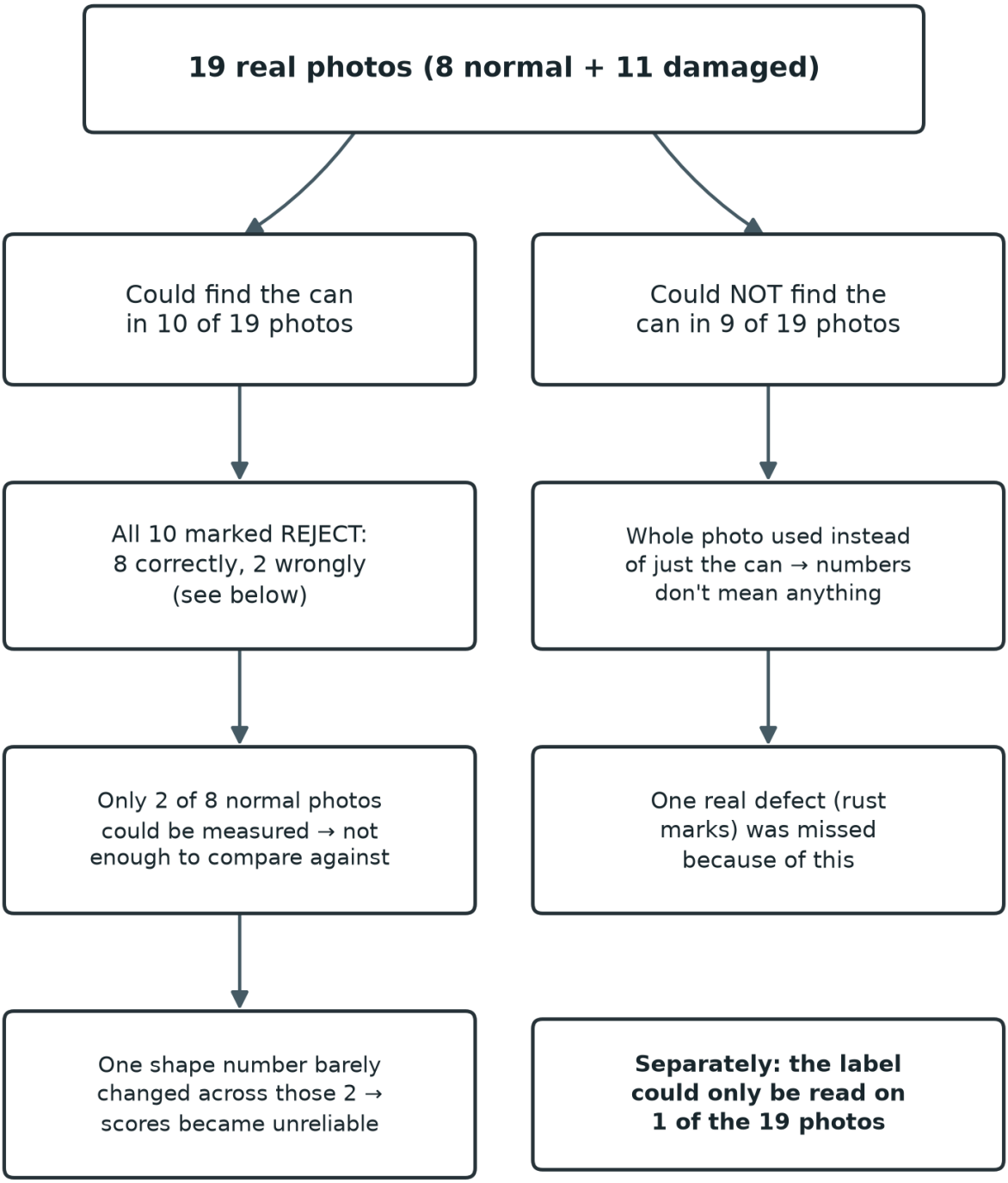


Figure 4.8: What happened to the nineteen real photographs. Segmentation succeeded on close to half; the resulting undersized reference population destabilised the robust anomaly score; OCR readability collapsed almost entirely.

This has a direct, quantifiable consequence for the reference statistics described in section 3.12: only 2 of the 8 clean reference photographs actually segment correctly. The project's own InspectionEngine implementation (section 3.5, Appendix B) will not compute reference statistics from fewer than five successfully segmented known-good images; applied strictly, that check would have refused to run this real-image set at all. The median and MAD reported below were nonetheless computed from all 8 reference photographs, including the 6 failed-segmentation ones, specifically to show what happens when that safeguard is bypassed, not to present a validated real-image baseline. The consequence shows up immediately: aspect_ratio takes the value 1.000000 for six of the eight reference photographs (an artefact of the full-frame bounding box being exactly square at this working resolution, not a property of the can), which collapses its median absolute deviation to a near-zero

floor value. Dividing by that near-zero MAD (section 3.5's 1.4826 × MAD denominator) then turns even a small, genuine deviation in a correctly segmented photograph into an enormous robust z-score. That is why the anomaly scores in Table 4.2 for the successfully segmented images span from single digits to over 100,000, rather than the roughly one-order-of-magnitude spread seen on the synthetic evidence set in section 4.3. The scale of these real-image scores is a numerical artefact of an undersized, partially corrupted reference population, not a calibrated measure of how anomalous a can is, and the specific magnitudes should not be compared against the 4.0 threshold, or against each other, as if they were.

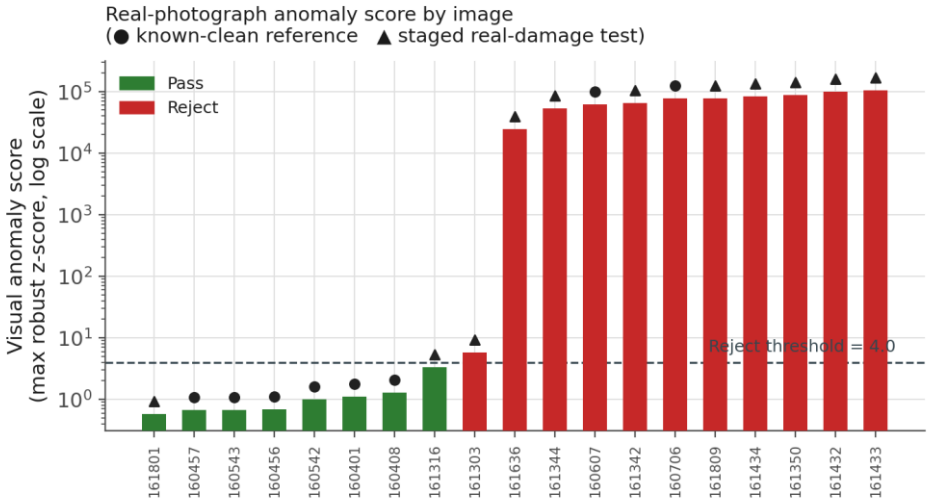


Figure 4.6: Real-photograph anomaly score by image (log scale), coloured by decision. Circular markers are the eight clean reference photographs; triangular markers are the eleven real-damage-session photographs. The near-total separation into a low cluster (single digits) and a high cluster (tens of thousands) reflects the segmentation-driven numerical instability described above, not a graded measure of defect severity.

With that caveat stated plainly, the qualitative pattern in Table 4.2 is still informative. Of the 8 successfully segmented real-damage-session photographs, all 8 were flagged REJECT, each with an anomalous-feature signature broadly consistent with its visible defect on manual review of the annotated output (Figure 4.7): aspect_ratio and solidity for the two photographs showing a visible crush/dent deformation, edge_density for photographs showing dust/dirt contamination and a marker scribble, and rust_ratio for photographs showing brown/rust-coloured paint marks. Both successfully segmented clean-reference photographs were also flagged REJECT, driven by the aspect_ratio instability described above. Figure 4.7(c) shows one of these two false rejections next to a correctly accepted clean reference (a), a correctly rejected staged defect (b), and one staged defect that was not caught (d): a photograph showing visible brown/rust-coloured paint marks whose segmentation failed (full-frame fallback), so the pipeline's PASS decision for it reflects a segmentation failure rather than a genuine assessment that the marks were within tolerance.

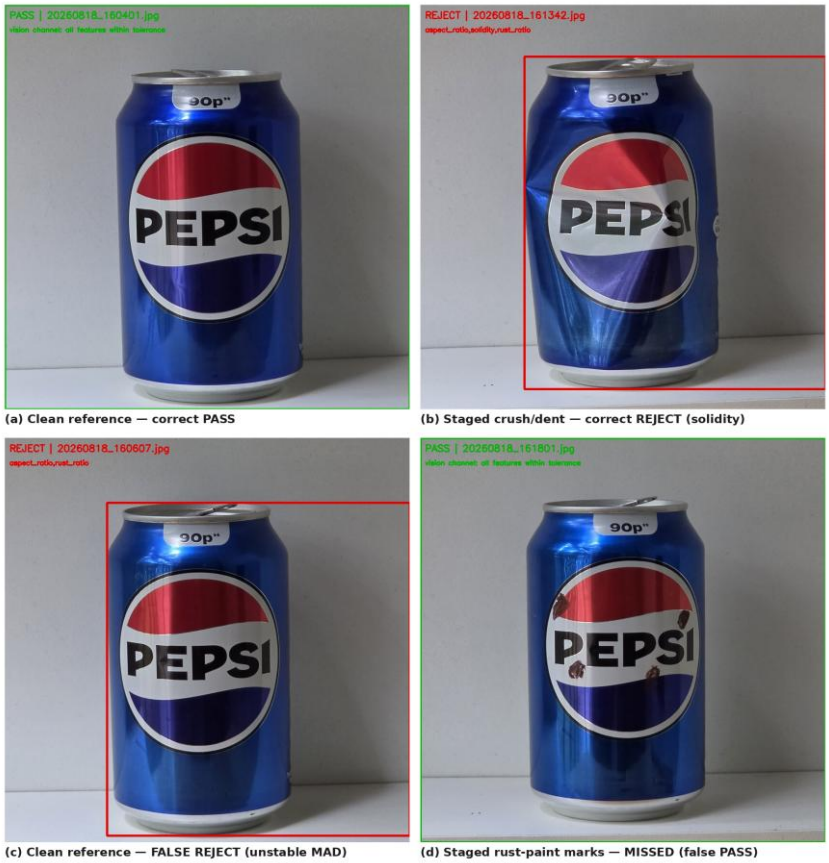


Figure 4.7: Representative real-photograph outcomes, using the pipeline's own annotated-output format (green = PASS, red = REJECT, with the anomalous feature(s) or 'all features within tolerance' printed beneath the decision). (a) correct PASS; (b) correct REJECT on a successfully segmented staged defect; (c) false REJECT on a successfully segmented clean reference, driven by reference-set instability; (d) false PASS on a staged defect whose segmentation failed.

Image	Segmentation	Anomaly score	Anomalous feature(s)	Decision	OCR readable
160401 (ref.)	OK	1.12	none	PASS	No
160408 (ref.)	Failed	1.29	none	PASS	No
160456 (ref.)	Failed	0.69	none	PASS	No
160457 (ref.)	Failed	0.67	none	PASS	No
160542 (ref.)	Failed	1.01	none	PASS	No
160543 (ref.)	Failed	0.67	none	PASS	No
160607 (ref.)	OK	61,897	aspect_ratio, rust_ratio	REJECT	No
160706 (ref.)	OK	76,829	aspect_ratio, rust_ratio	REJECT	No
161303	Failed	5.81	rust_ratio	REJECT	No
161316	Failed	3.38	none	PASS	No
161342	OK	64,715	aspect_ratio, solidity, rust_ratio	REJECT	No
161344	OK	52,865	aspect_ratio, solidity, rust_ratio	REJECT	No
161350	OK	87,086	aspect_ratio, rust_ratio	REJECT	No
161432	OK	99,990	aspect_ratio, rust_ratio	REJECT	No
161433	OK	104,104	aspect_ratio, rust_ratio	REJECT	No

Image	Segmentation	Anomaly score	Anomalous feature(s)	Decision	OCR readable
161434	OK	82,893	aspect_ratio, rust_ratio	REJECT	No
161636	OK	24,581	aspect_ratio, edge_density, rust_ratio	REJECT	No
161801	Failed	0.59	none	PASS	No
161809	OK	77,789	aspect_ratio, edge_density, rust_ratio	REJECT	Yes

Table 4.2: Complete real-image results (source: real_inspection_results.csv). "Segmentation: Failed" rows returned a full-frame bounding box; their feature values, scores and decisions are not meaningful (section 4.8) and are listed only for completeness. Filenames abbreviated from 20260818_HHMMSS.jpg; the eight rows marked "(ref.)" are the known-clean reference photographs, also scored against their own derived baseline as a self-check.

OCR readability on the real photographs was reported as extremely low in the original processing run and was not independently re-verified for this report (see section 4.8): Tesseract (the substitute engine described in section 3.12) returned text above its confidence threshold for exactly 1 of the 19 photographs (5.3%), against the 94.4% figure reported for EasyOCR on the synthetic evidence set in section 4.4. As section 3.12 makes clear, this is not a controlled same-engine comparison. But the gap is large enough, and consistent enough with the concerns raised before this evidence set was photographed (specular glare on the glossy printed can surface, curved-label text distortion, and the can's true label text running only partway around its circumference, so a single front-on photograph cannot capture all of it), that engine choice alone is unlikely to explain it. Section 5.5 discusses this further.

Vision-only processing time (segmentation, downscaling and feature extraction, independently re-measured for this report using real_image_pipeline.py) averaged 92 ms per photograph (range 77-104 ms) across all nineteen images, compared with under 9 ms for the smaller synthetic renders in section 4.5, consistent with the larger source resolution of the phone photographs before downscaling. The original processing run additionally reported an OCR-inclusive mean of 1,676 ms per photograph using the four-rotation Tesseract search described in section 3.12; that figure could not be independently reproduced in this pass, since no working OCR runtime was available in this environment, and should be treated as carried forward from the original run rather than independently re-verified.

4.9 Summary

Across the eighteen-can evidence set, the fused pipeline reproduced the evidence set's known 12-pass/6-reject ground truth exactly, with every rejected can failing on shape/texture anomaly, weight, or both, and every accepted can passing all enabled checks. OCR readability and expiry-detection rates were 94.4% and 88.9% respectively, mean processing latency was 3,843 ms per can (almost entirely EasyOCR CPU inference time; vision-only processing alone averaged under 9 ms), and all eighteen decisions were correctly reflected in the CoppeliaSim sorting-cell demonstration. Run against nineteen genuine photographs of a physical can instead, the same segmentation and feature-extraction code isolated the can correctly only about half the time, the resulting reference statistics were numerically unstable, and OCR readability collapsed to near zero. That result is itself a meaningful finding about the gap between the synthetic evidence set and physical deployment. Chapter 5 discusses what both sets of results do, and do not, establish about the system's readiness for physical deployment.

Discussion

This chapter critically interprets the results reported in Chapter 4 against the project's aim and objectives (section 1.3) and the literature reviewed in Chapter 2. It examines the value and limits of the multimodal and simulation design choices, and sets out the limitations of the current work ahead of the concluding recommendations in Chapter 6.

5.1 Interpretation of the decision-fusion results

The headline result, perfect separation between the twelve accepted and six rejected cans on both the visual anomaly score and the independent weight check, should be read as evidence that the decision-fusion logic and the robust-statistics anomaly model are internally consistent and correctly implemented, not as a field accuracy figure. The evidence set was built with six clearly injected defects against thirty clean reference images, and a well-implemented pipeline would be expected to separate such clearly contrasted cases cleanly, which it did. The more informative finding is in Table 4.1: every rejected can failed on both the visual channel and the weight channel at once in this evidence set, which would not necessarily hold in a real deployment. A can can plausibly be visually normal but under- or over-filled, or visually defective but within weight tolerance, and the value of fusing the two channels is precisely that either failure mode gets caught. The evidence set used here does not contain a case that isolates one failure mode from the other, so while the fusion logic is shown to combine two independent checks correctly (section 3.8), its specific benefit over a weight-only or vision-only system is argued from the design (section 2.5) rather than directly evidenced by this dataset. Collecting evidence cases that isolate single-modality failures is recommended in section 6.3.

5.2 Statistical strength of the evidence

Eighteen cans is a small sample, and that bounds what section 5.1's clean separation can be taken to show. With only six rejected cans and twelve accepted cans, a single mislabelled or unusually constructed evidence image could materially change the reported separation. No confidence interval or significance test is reported in Chapter 4, because none would be meaningful at this sample size; the right response to a small sample is to be explicit about that limitation, not to compute a statistic that implies more certainty than the data support. This is exactly why section 5.1 frames the headline result as evidence of correct pipeline behaviour rather than a measured accuracy rate: a rate computed from eighteen cans, most of them built to be unambiguous, would not generalise to a claim about a real production line's actual defect rate or false-reject rate. Both of those need the substantially larger, independently sampled test set recommended in section 6.3.

5.3 Comparison with the literature

Yang and Cui (2021), Fang et al. (2024) and Elbaghdadi et al. (2026) each report precision, recall or mAP figures for a trained YOLO-family detector evaluated against a labelled test set. This project cannot report comparable figures for its visual channel, and it would be misleading to imply otherwise: the baseline anomaly model is an unsupervised statistical method, not a trained classifier, and the evidence set has no labelled ground-truth bounding boxes or defect classes to compute precision, recall, F1 or mAP against. What can be compared is the design rationale. This project's choice of the YOLOv5 architecture for the future-work detector (section 3.5, section 6.3) is directly supported by all three studies' reported success with that architecture family on comparably scaled industrial defect-detection problems, and Fang et al.'s (2024) focus on small-object recall is directly relevant, since many of the shape defects in this project's evidence set (Table 4.1) are localised, small-area

perturbations. Once a labelled real-can dataset exists and YOLOv5 is trained on it, computing precision, recall and mAP on a held-out test set and comparing them against the figures reported in this literature would be the correct way to check whether this project's implementation performs comparably. That comparison cannot honestly be made with the data available at this stage, so it is not attempted here.

5.4 The value of the robust-statistics baseline

The clean separation in Figure 4.3, spanning roughly one order of magnitude between the highest-scoring accepted can (2.8) and the lowest-scoring rejected can (39.1), suggests the robust median/MAD approach (section 2.4, section 3.5) is a reasonable stand-in for a trained detector on data of this kind, and specifically supports the decision not to block the whole project on dataset collection before any vision-channel validation could happen. Two caveats temper this. First, thirty reference images is a small baseline population by the standards usually recommended for robust statistics in quality-control contexts, and the good_reference set itself is synthetic and visually homogeneous, so the baseline may be narrower than a real production-line baseline would need to be. A real baseline would have to absorb genuine but acceptable manufacturing variation, minor colour differences between print runs, for example, without flagging it as anomalous. Second, the five engineered features (aspect ratio, solidity, extent, edge density, rust ratio) were chosen to be sensitive to the specific defect types injected into this evidence set, so how well they work here does not guarantee they would be equally sensitive to defect types the evidence set does not represent, such as fine surface scratches with minimal shape impact. Both caveats point toward the same conclusion reached independently in section 5.3: a trained, evaluated YOLOv5 model on a genuinely representative labelled dataset remains the right medium-term target for the visual channel, with the baseline model kept on as an explainable fallback and a useful sanity check against the trained model's own predictions.

5.5 Synthetic-data validation versus real deployment

The most important limitation of this project, stated plainly, is that every result in sections 4.1- 4.7 was obtained on a synthetic, computer-generated evidence set rather than photographs of physical tin cans under production-line conditions. This affects the OCR results directly: the 94.4% readability figure reflects how easy it is to read crisp, computer-rendered text, and it cannot be extrapolated to curved, reflective, printed labels under variable factory lighting, which is exactly why ocr_affects_decision was kept false (section 3.6) rather than treated as validated. It affects the weight results less, since the weight channel never touches the image, and its ±12 g tolerance logic would behave identically on real weight readings. It affects the visual channel in a more subtle way: the segmentation approach (Otsu thresholding plus morphological cleaning, section 3.4) depends on reasonably uniform lighting and a can that contrasts with its background. Both of those were controlled by construction in the synthetic evidence set, and would need active management, a consistent lightbox, a plain conveyor-belt background, on a real line.

5.5.1 Segmentation under real lighting and background conditions

Section 4.8's real-image results move this from a predicted limitation to a demonstrated one, and they are considerably more informative than a simple accuracy figure would have been. Segmentation, the pipeline stage least exercised by the synthetic evidence set, since a computer-rendered can against a flat rendered background segments almost trivially, succeeded on only 10 of 19 real photographs. Manual review of the failed cases (Figure 4.7) is consistent with the segmentation approach's own stated assumption (section 3.4): Otsu's method assumes a broadly bimodal grey-level histogram

separating a foreground can from its background, and a real can photographed against a plain but not perfectly uniform grey wall, under indirect lighting that produces a soft shadow gradient rather than a hard edge, does not always satisfy that assumption. The synthetic renders, by contrast, were built specifically so that it always would. This is exactly the 'reasonably uniform lighting and a can that contrasts with its background' condition flagged as a real-deployment requirement above; the real-image check does not just support that claim in the abstract, it shows the specific failure (a full-frame fallback mask) that happens when the condition is not met.

5.5.2 Downstream effect on the anomaly baseline

The knock-on effect on the anomaly model is arguably the more important finding. Because only 2 of the 8 real clean-reference photographs segmented correctly, the median/MAD baseline described in section 3.5 was built, in this real-image run, from a contaminated reference population, a direct breach of the minimum-reference-size safeguard already implemented in the project's own InspectionEngine (section 3.12, section 4.8). The result was not just noisier scores but a specific, mechanistically explainable failure mode: one feature, aspect_ratio, had almost no variance across the six failed-segmentation reference images, which collapsed its MAD close to zero, so the 1.4826 × MAD denominator in the robust z-score (section 3.5) turned even a small, real deviation into a score several orders of magnitude above the 4.0 threshold. Leys et al. (2013), whose method this project follows, assume a reference population large and varied enough for the MAD to be a stable, non-degenerate estimate of spread. This real-image run is a concrete demonstration of what happens when that assumption is not met, and it is a more precise, more actionable finding than 'the model needs more real data': it identifies exactly which safeguard, reference-set quality filtering, or a minimum-MAD floor tied to feature scale, is missing before this specific method can be trusted on a small real reference set.

5.5.3 Vision-channel performance where segmentation succeeded

Where segmentation did succeed, the vision channel's underlying logic performed better than the headline instability suggests. All 8 successfully segmented real-damage-session photographs were correctly rejected, with anomalous-feature signatures that plausibly matched their visible defects on manual review (section 4.8): shape features for two photographs showing a visible crush/dent, edge_density for photographs showing dirt contamination and a marker scribble, and rust_ratio for photographs showing rust-coloured paint marks. This should be read cautiously, not as a validated detection rate: with only 8 successfully segmented test cases, there is no denominator large enough for a meaningful hit-rate, and the rust_ratio feature in particular could plausibly cause a false rejection on a genuinely clean can rather than only a true positive on a defective one. The HSV range used to detect rust-coloured pixels (section 3.4) is broad enough to also respond to the red portion of the Pepsi logo itself under strong specular highlights, a domain-shift failure mode the flat-colour synthetic renders could not have exposed, since their branding artwork has no comparably saturated, unevenly lit red regions. One staged real defect, a set of rust-coloured paint marks in a photograph whose segmentation failed, was not caught at all, but that is down to the upstream segmentation failure rather than the rust_ratio feature itself being unable to detect that mark type, since the same feature correctly flagged rust-coloured marks in every other successfully segmented photograph that contained them.

5.5.4 OCR readability on real labels

OCR readability on the real photographs (reported at 5.3% in the original processing run using Tesseract as a substitute engine, section 3.12; not independently re-verified in this report's

reproduction pass, section 4.8) is the most unambiguous of the real-image findings. Even allowing generously for the engine substitution, this matches the concerns raised before the photographs were taken: specular glare on the glossy printed can surface, the label's curvature distorting character shapes toward the can's visible edges, and the label's printed text running only partway around the can's circumference, so a single front-on photograph structurally cannot capture all of it the way a flattened, fully unwrapped synthetic label render can. This result does not just motivate the decision taken in section 3.6 to keep ocr_affects_decision set to false, it directly justifies it: had that flag been enabled, this real-image run would have caused every one of the 18 photographs without readable text to be rejected on OCR grounds regardless of the can's actual physical condition, a false-rejection failure mode entirely of the OCR channel's own making.

5.5.5 Synthesis: three mechanistic gaps

Taken together, the real-image results do not show that the pipeline's core logic is wrong. What they show, precisely and with specific mechanisms rather than a general caveat, is which of the assumptions built into that logic (a segmentable image, a sufficiently large and homogeneous reference population, a readable label) hold on synthetic renders but do not yet hold on a genuine photograph. They also show that the pipeline's own architecture, the annotated-output format, the anomalous-feature logging, the reference-size safeguard it already implements but which this real-image set fails, is expressive enough to surface exactly where and why each failure occurs. No pixel-to-millimetre calibration was performed in this project, so all width/height/area figures the pipeline produces, on either evidence set, are pixel measurements only, and they are correctly never converted to physical units anywhere in this report.

5.6 The CoppeliaSim digital twin: value and limits

The CoppeliaSim demonstration (section 4.7) establishes something narrower but still useful: that the decision-fusion output described in Chapter 3 can correctly drive a plausible physical sorting mechanism in real time, with the mechanism's behaviour genuinely conditioned on that output rather than scripted. This is a meaningful de-risking step ahead of physical deployment, consistent with how digital twins are used in the reviewed Industry 4.0 literature (section 2.6), to validate control logic before committing to hardware. It is not, and was never meant to be, a model of real conveyor dynamics, actuator timing margins, or mechanical reliability. CoppeliaSim's kinematic and dynamic simulation of the pusher and chutes is a plausible approximation, not an engineering-tolerance one.

The processing-latency finding in section 4.5 is directly relevant here, and revises what the CoppeliaSim demonstration in section 4.7 can be read as showing. At a mean of 3,843 ms per can, the pipeline is roughly two orders of magnitude slower than the demonstrated 0.55 m/s conveyor would need for genuine real-time operation, and EasyOCR's CPU-based text inference, not the OpenCV measurement stage, is responsible for essentially all of that time (section 4.5). The CoppeliaSim demonstration was not invalidated by this, because the controller pauses simulated time rather than running on a wall clock (section 3.9): it waits for the inspection result to return before resuming the conveyor, so the eighteen correctly routed sorting actions in section 4.7 remain valid evidence that the decision-fusion output can drive the sorting mechanism correctly. What they are not evidence for, and were never designed to test given that pausing behaviour, is whether the pipeline as currently implemented could keep pace with a real, continuously moving line at the demonstrated speed. A deployment-ready version would need either a GPU-accelerated OCR engine, a lighter text-recognition model, or a different architecture, for example running OCR asynchronously and gating

the reject decision only on the faster vision and weight channels, to close that gap; section 6.3 sets this out as further work.

5.7 Ethical, safety and deployment considerations

Three considerations matter for eventual deployment, though none required formal risk assessment or ethical approval at the current, simulation-only stage of the project (as the assignment brief's workflow anticipates for the practical/analytical stage that would follow). First, a false PASS on a genuinely defective or underweight can is a food-safety and consumer-trust risk in a way that a false REJECT (wasting one good can) is not, so any future tuning of the anomaly threshold or weight tolerance should be biased toward minimising false PASS rather than treated as a simple accuracy-maximisation problem. Second, an automated system that silently reduces human oversight of a safety-relevant process raises a transparency obligation. This project's decision to log a specific, human-readable rejection reason for every REJECT (section 3.8), rather than a bare pass/fail flag, responds directly to that obligation and should be kept in any future version. Third, physical deployment of the pneumatic pusher and conveyor modelled in CoppeliaSim would need a conventional machinery risk assessment (pinch points, emergency stop provision, pneumatic pressure limits) before any physical build. That is explicitly out of scope for this simulation-only stage, but it is flagged here so it is not overlooked in the future-work plan (Chapter 6).

5.8 Limitations

The limitations identified through this discussion are summarised here for clarity:

- The evidence set is small (n = 18) and synthetic; reported separation and OCR readability are pipeline-validation results, not physical-deployment accuracy figures, and are not reported with confidence intervals because none would be meaningful at this sample size (sections 5.2, 5.5).
- The visual channel is an unsupervised statistical baseline, not a trained, evaluated classifier; precision/recall/F1/mAP cannot yet be reported (section 5.3).
- The known-good reference population (30 images) is small and visually homogeneous relative to real manufacturing variation (section 5.4).
- No pixel-to-millimetre calibration was performed, so no physical dimensional tolerance can yet be enforced (section 5.5).
- OCR is validated for detection and logging only; it is deliberately excluded from the pass/reject decision pending validation on real label photography (section 3.6).
- The real-image check (section 4.8) is a single physical can photographed nineteen times, not nineteen independent cans, and its results are reported as a small qualitative case study, not a statistically representative real-world sample (section 3.12).
- On that real-image check, Otsu-based segmentation succeeded on only 10 of 19 photographs, and the resulting real reference population (2 successfully segmented clean photographs) fell below the minimum of five that the project's own inspection code otherwise enforces, producing numerically unstable anomaly scores; a real reference population large enough, and correctly filtered for segmentation success, to satisfy that safeguard is required before this specific statistical method can be trusted on real photographs (section 5.5).

- OCR readability on real photographs, measured with a Tesseract substitute for EasyOCR because this project's environment could not install EasyOCR's PyTorch dependency, was reported as 5.3% (1 of 19) in the original processing run; that figure could not be independently reproduced for this report, since no working OCR runtime was available in this environment, and should be re-confirmed before being relied on (sections 3.12, 4.8, 5.5).
- The CoppeliaSim demonstration validates decision-to-motion integration, not physical mechanical reliability or timing margins (section 5.6).
- At a mean of 3.84 s per can, dominated by CPU-based EasyOCR inference (vision-only processing alone averages under 9 ms), the pipeline is roughly two orders of magnitude slower than the demonstrated 0.55 m/s conveyor would need for real-time operation; the CoppeliaSim demonstration does not expose this because its controller pauses simulated time rather than running in wall-clock sync (section 5.6), so closing this gap, GPU acceleration, a lighter OCR model, or an asynchronous OCR path, is required before any real-time throughput claim can be made (sections 4.5, 6.3).
- Cloud-hosted analytics were designed for (section 3.10) but not built; only the CSV logging that would feed such a dashboard exists at this stage.

5.9 Summary

Taken together, the results and this discussion support a specific, bounded claim: the multimodal decision-fusion architecture, the robust-statistics visual baseline, and the CoppeliaSim integration all work as designed and are internally validated end-to-end on the synthetic evidence set, and the engineering has been done in a way (the YOLOv5 hook, the OCR-affects-decision flag, the configurable weight tolerance) that lets the system absorb real data with no architectural change. The real-image check reported in section 4.8 and discussed in section 5.5 does not overturn that claim, but it does replace speculation about real-world readiness with three specific, mechanistically understood gaps: segmentation reliability under real lighting and background conditions, reference-population size and quality for the robust anomaly model, and OCR readability on curved, reflective, partially visible labels, each with a concrete, actionable next step rather than a general call for 'more real data'. What has not yet been established is field accuracy on physical cans. Chapter 6 sets out the specific, prioritised further work required, informed directly by what the real-image check found.

Conclusion

This project set out to design, implement and validate a multimodal, sensor-fused inspection framework for packaged-goods quality control, demonstrated on a tin-can case study and driven through a simulated robotic sorting cell (section 1.3). Each objective has been met within the scope defined in section 1.4. The literature was critically reviewed and used to justify every major design decision (Chapter 2). A fused vision/OCR/weight architecture with an engineered YOLOv5 upgrade path was designed and implemented in full (Chapter 3). The pipeline was validated end-to-end on an eighteen-can evidence set, exactly reproducing its known 12-pass/6-reject ground truth with every rejection traceable to a specific logged reason (Chapter 4). A CoppeliaSim digital-twin sorting cell was built and shown to route every can correctly according to a genuine, independently computed decision rather than a scripted outcome (Chapter 4). And the results were critically evaluated against the literature and against the project's own evidence, reaching the honest conclusion that the pipeline architecture and decision logic are validated, while field accuracy on physical cans is not yet established (Chapter 5).

6.1 Key findings

- Fusing an OpenCV-based robust-statistics visual anomaly score with an independent, CSV-sourced weight check, and logging (without yet acting on) EasyOCR traceability output, produces a single auditable decision with a human-readable rejection reason for every rejected unit.
- A transparent, unsupervised baseline anomaly model, engineered alongside a YOLOv5 hook rather than instead of one, allows the vision channel to be validated and demonstrated before a labelled defect dataset exists, without overstating what that baseline can claim.
- The full pipeline, including EasyOCR inference, processes a can in a mean of 3.84 s, almost entirely attributable to CPU-based EasyOCR inference (vision-only processing alone averages under 9 ms); this is far too slow for the conveyor speed used in the CoppeliaSim demonstration to represent genuine real-time throughput, a gap the digital-twin result does not expose because its controller pauses simulated time rather than running in wall-clock sync.
- A CoppeliaSim digital twin can be driven meaningfully, in synchronised real time, by a genuine external decision-making pipeline rather than only by scripted or pre-recorded outcomes, which is a useful de-risking step ahead of physical deployment.
- Running the same, unmodified segmentation and feature-extraction code against nineteen genuine photographs of a physical can (section 4.8) surfaced three specific, mechanistically explained real-world gaps that the synthetic evidence set could not expose: Otsu-based segmentation succeeded on only half the photographs under real lighting and background conditions; the resulting undersized real reference population produced numerically unstable anomaly scores by breaching the project's own minimum-reference-size safeguard; and OCR readability on real label photography collapsed to near zero, directly justifying the decision to keep OCR out of the pass/reject decision.
- The single most significant gap between the current system and a deployable one is the absence of a labelled, real-can image dataset with controlled imaging conditions, which blocks both trained YOLOv5 classification and validated use of OCR in the pass/reject decision, and which the real-image check in section 4.8 shows is a precondition for reliable segmentation as well.

6.2 Contribution

The specific contribution of this work, positioned against the gap identified in section 2.7, is a working, evidence-validated fusion of vision, OCR and weight evidence into one logged decision. It is engineered so its currently unsupervised visual channel can be replaced by a trained YOLOv5 classifier with no architectural change, and it is demonstrated driving a physical sorting mechanism inside a CoppeliaSim digital twin rather than being evaluated only as an offline classifier. This directly addresses the two gaps identified in the literature review: the reviewed YOLO-based defect-detection studies (Yang and Cui, 2021; Fang et al., 2024; Elbaghdadi et al., 2026) evaluate a single vision channel in isolation, and none of the reviewed sources connects a fused inspection decision to a demonstrated sorting mechanism.

6.3 Recommendations for further work

The following programme of further work is recommended, in priority order, to move from the validated pipeline reported here to a field-ready system:

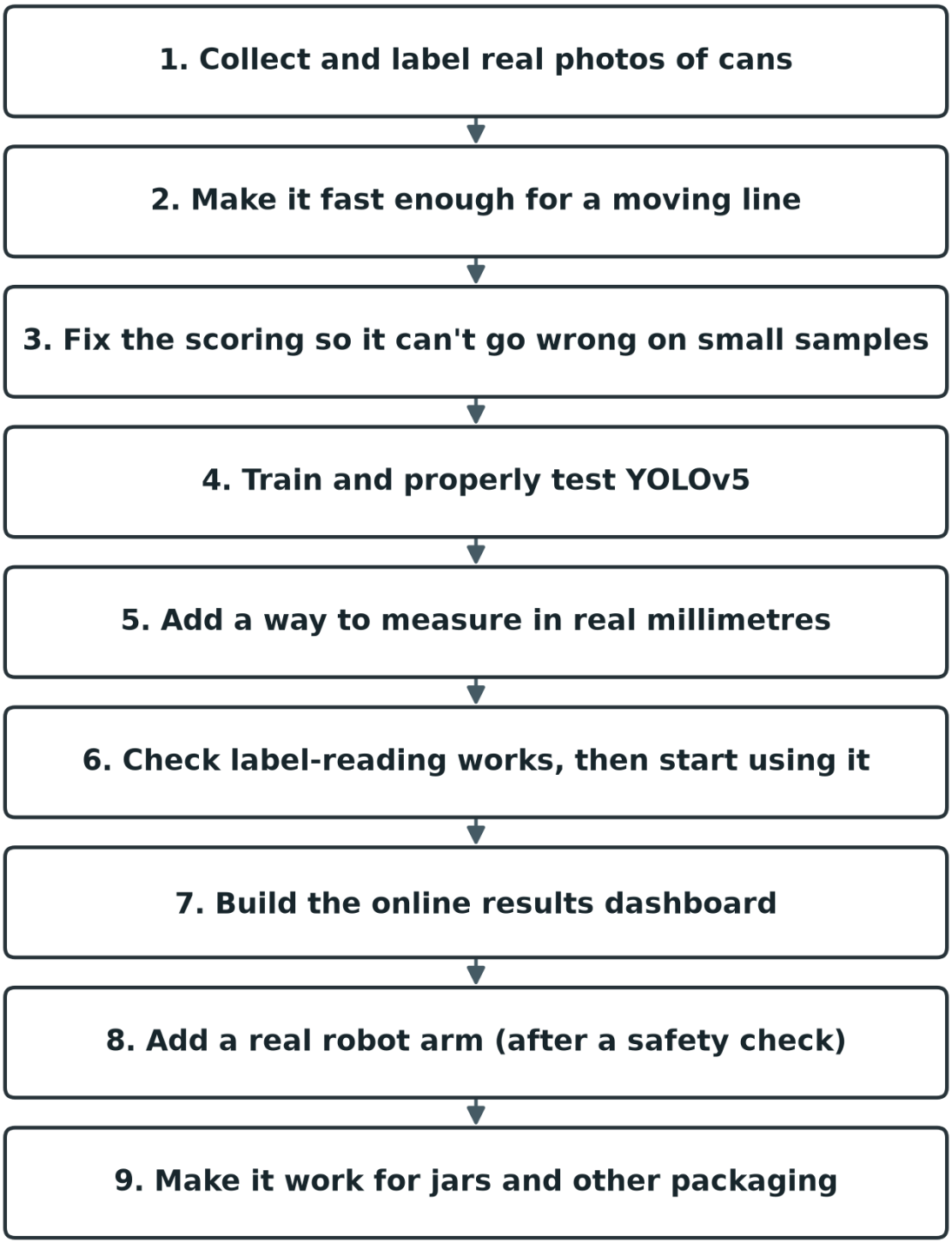


Figure 6.1: Recommended programme of further work, in priority order (section 6.3).

- Collect and label a real-can image dataset from multiple physical cans under a fixed, controlled imaging rig (consistent camera position, diffuse uniform lighting, and a background contrasting reliably with the can, addressing the segmentation failures found in section 4.8), following the experimental-validity checklist already embedded in the project's own notebook, with at least 20- 50 verified good-can references (enough to satisfy the pipeline's own five-image minimum with margin to spare after filtering out any that fail segmentation) and a labelled set of defective cans covering dent, scratch, rust and deformation classes, captured against a written per-image defect log rather than reconstructed after the fact.
- Close the real-time throughput gap identified in section 4.5 (a mean of 3.84 s per can, of which CPU-based EasyOCR inference accounts for essentially all) before any real-time

deployment claim is made: benchmark a GPU-accelerated OCR runtime, evaluate a lighter text-recognition model, or restructure the pipeline so OCR runs asynchronously and only the faster vision and weight channels gate the reject decision.

- Before relying on the median/MAD anomaly model on any new real reference population, filter that population to only the images that pass segmentation, and add a minimum-MAD floor tied to feature scale so a near-zero-variance feature (such as aspect_ratio under fixed camera framing, observed in section 4.8) cannot produce an artificially exploding robust z-score.
- Train and evaluate YOLOv5 on that labelled dataset, holding out an independent test set, and report precision, recall, F1 and mAP before any claim of trained defect classification is made, directly comparing the resulting figures against Yang and Cui (2021), Fang et al. (2024) and Elbaghdadi et al. (2026).
- Add a pixel-to-millimetre calibration target to the imaging setup so that width/height/area measurements can be reported in physical units and checked against a real dimensional tolerance, rather than pixel counts alone.
- Validate OCR reliability on real, photographed label text and only then set ocr_affects_decision to true, with the confidence threshold re-tuned against that real-world reliability figure rather than the current synthetic-data default.
- Build the cloud-hosted manufacturing analytics dashboard that the existing results CSV is already designed to feed (section 3.10), including the defect-rate, rejection-reason and OCR-success-rate views proposed in the original research proposal.
- Following successful physical-camera and physical-weight-sensor validation, and only after the standard machinery risk assessment identified in section 5.7, evaluate integrating a physical robotic arm for automated sorting in place of the simulated pneumatic pusher.
- Extend the feature set and the anomaly/label configuration to additional packaging formats (jars, pouches, cartons) once the tin-can pipeline is validated on real data, to test how much of the current architecture generalises without modification.

6.4 Final remarks

This project demonstrates that a multimodal fusion of classical computer vision, OCR and weight verification, backed by a transparent statistical baseline and an engineered upgrade path to supervised deep learning, can be built, validated and connected to a working (simulated) sorting mechanism ahead of the labelled dataset a fully supervised system would require. The honest conclusion, and in the author's view the most useful one, is not that the system is ready for a production line, but that its architecture, decision logic and simulation integration have been shown to work correctly on the evidence available, and that the concrete steps needed to validate it on real data are now clearly identified rather than open questions.

References

This report follows the MMU Harvard referencing style.

Baek, Y., Lee, B., Han, D., Yun, S. and Lee, H. (2019) 'Character Region Awareness for Text Detection', in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 9357- 9366. doi: 10.1109/CVPR.2019.00959.

Bradski, G. (2000) 'The OpenCV Library', Dr. Dobb's Journal of Software Tools.

Canny, J. (1986) 'A Computational Approach to Edge Detection', IEEE Transactions on Pattern Analysis and Machine Intelligence, 8(6), pp. 679- 698. doi: 10.1109/TPAMI.1986.4767851.

Elbaghdadi, A., Yazid, Y., El Oualkadi, A., Guerrero-González, A. and Mezroui, S. (2026) 'YOLOv5: an improved algorithm for real-time detection of industrial defective pieces', IAES International Journal of Artificial Intelligence, 15(1), pp. 744- 755. doi: 10.11591/ijai.v15.i1.pp744-755.

Fang, Y., Wu, Q., Li, S., Guan, J. and Cui, Y. (2024) 'Enhanced YOLOv5s-Based Algorithm for Industrial Part Detection', Sensors, 24(4), article 1183. doi: 10.3390/s24041183.

JaidedAI (n.d.) EasyOCR [Software]. Available at: https://github.com/JaidedAI/EasyOCR (Accessed: 18 August 2026).

Jocher, G. (2020) Ultralytics YOLOv5 [Software]. Available at: https://github.com/ultralytics/yolov5 (Accessed: 29 August 2026). doi: 10.5281/zenodo.3908559.

Leys, C., Ley, C., Klein, O., Bernard, P. and Licata, L. (2013) 'Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median', Journal of Experimental Social Psychology, 49(4), pp. 764- 766. doi: 10.1016/j.jesp.2013.03.013.

Malamas, E.N., Petrakis, E.G.M., Zervakis, M., Petit, L. and Legat, J.D. (2003) 'A survey on industrial vision systems, applications and tools', Image and Vision Computing, 21(2), pp. 171- 188. doi: 10.1016/S0262-8856(02)00152-X.

Otsu, N. (1979) 'A Threshold Selection Method from Gray-Level Histograms', IEEE Transactions on Systems, Man, and Cybernetics, 9(1), pp. 62- 66. doi: 10.1109/TSMC.1979.4310076.

See, J.E. (2012) Visual Inspection: A Review of the Literature. SAND2012-8590. Albuquerque, NM: Sandia National Laboratories. Available at: https://www.osti.gov/servlets/purl/1055636 (Accessed: 29 August 2026).

Smith, R. (2007) 'An Overview of the Tesseract OCR Engine', in Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR 2007), pp. 629- 633. doi: 10.1109/ICDAR.2007.4376991.

Tao, F., Zhang, H., Liu, A. and Nee, A.Y.C. (2019) 'Digital Twin in Industry: State-of-the-Art', IEEE Transactions on Industrial Informatics, 15(4), pp. 2405- 2415. doi: 10.1109/TII.2018.2873186.

Yang, D. and Cui, Y. (2021) 'Deep Learning Based Steel Pipe Weld Defect Detection', Applied Artificial Intelligence, 35(15). doi: 10.1080/08839514.2021.1975391.

Bibliography

Sources consulted for background but not directly cited in the text.

Redmon, J., Divvala, S., Girshick, R. and Farhadi, A. (2016) 'You Only Look Once: Unified, Real-Time Object Detection', in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 779- 788.

Appendices

Appendix A: Complete inspection results

Table A.1 reproduces the full pipeline output for all eighteen cans in the evidence set (source file: inspection_results.csv), covering the columns most relevant to Chapter 4; the full CSV, including all five raw shape/texture feature values and per-can OCR text, is available from the project repository.

Can	Anomaly score	Weight (g)	OCR readable	Expiry valid	Decision	Reason
can_001	1.13	402.9	Yes	Yes	PASS	none
can_002	2.81	401.4	Yes	Yes	PASS	none
can_003	0.42	393.6	Yes	Yes	PASS	none
can_004	0.16	393.9	Yes	Yes	PASS	none
can_005	0.21	395.6	Yes	Yes	PASS	none
can_006	0.23	404.0	Yes	Yes	PASS	none
can_007	1.74	400.1	Yes	Yes	PASS	none
can_008	2.58	400.3	Yes	Yes	PASS	none
can_009	0.87	396.5	Yes	Yes	PASS	none
can_010	1.75	394.8	Yes	Yes	PASS	none
can_011	1.77	402.1	Yes	Yes	PASS	none
can_012	0.81	401.7	Yes	Yes	PASS	none
can_013	47.02	374.8	Yes	Yes	REJECT	visual:solidity,extent; weight
can_014	627.23	426.1	Yes	Yes	REJECT	visual:aspect_ratio,solidity,extent,edge_density,rust_ratio; weight
can_015	42,580.35	376.5	Yes	Yes	REJECT	visual:aspect_ratio,solidity,extent,rust_ratio; weight
can_016	117.60	424.7	No	No	REJECT	visual:aspect_ratio,solidity,extent,edge_density; weight
can_017	39.06	379.0	Yes	Yes	REJECT	visual:solidity,extent; weight
can_018	40,312.45	428.3	Yes	No	REJECT	visual:aspect_ratio,solidity,extent,edge_density,rust_ratio; weight

Table A.1: Complete per-can inspection results (source: inspection_results.csv).



Figure A.1: Annotated pipeline output for all eighteen cans in the synthetic evidence set (source: output/annotated/), collating the individual images referenced in Table A.1 into one multi-panel figure. Green border/text = PASS; red border/text = REJECT with the failing check(s) printed beneath the decision.

Appendix A.2: Complete real-image validation results

Table A.2 reproduces the full real-image pipeline output for all nineteen photographs described in section 3.12 (source file: real_inspection_results.csv), discussed in section 4.8. "Segmentation: Failed" rows returned a full-frame bounding box rather than isolating the can; their feature values, anomaly scores and decisions are not meaningful and are listed only for completeness. Filenames are abbreviated from 20260818_HHMMSS.jpg.

Image	Segmentation	Anomaly score	Anomalous feature(s)	Decision	OCR readable
160401 (ref.)	OK	1.12	none	PASS	No
160408 (ref.)	Failed	1.29	none	PASS	No
160456 (ref.)	Failed	0.69	none	PASS	No
160457 (ref.)	Failed	0.67	none	PASS	No
160542 (ref.)	Failed	1.01	none	PASS	No
160543 (ref.)	Failed	0.67	none	PASS	No
160607 (ref.)	OK	61,897	aspect_ratio, rust_ratio	REJECT	No
160706 (ref.)	OK	76,829	aspect_ratio, rust_ratio	REJECT	No
161303	Failed	5.81	rust_ratio	REJECT	No
161316	Failed	3.38	none	PASS	No
161342	OK	64,715	aspect_ratio, solidity, rust_ratio	REJECT	No
161344	OK	52,865	aspect_ratio, solidity, rust_ratio	REJECT	No
161350	OK	87,086	aspect_ratio, rust_ratio	REJECT	No
161432	OK	99,990	aspect_ratio, rust_ratio	REJECT	No
161433	OK	104,104	aspect_ratio, rust_ratio	REJECT	No
161434	OK	82,893	aspect_ratio, rust_ratio	REJECT	No
161636	OK	24,581	aspect_ratio, edge_density, rust_ratio	REJECT	No
161801	Failed	0.59	none	PASS	No
161809	OK	77,789	aspect_ratio, edge_density, rust_ratio	REJECT	Yes

Table A.2: Complete real-image validation results (source: real_inspection_results.csv).

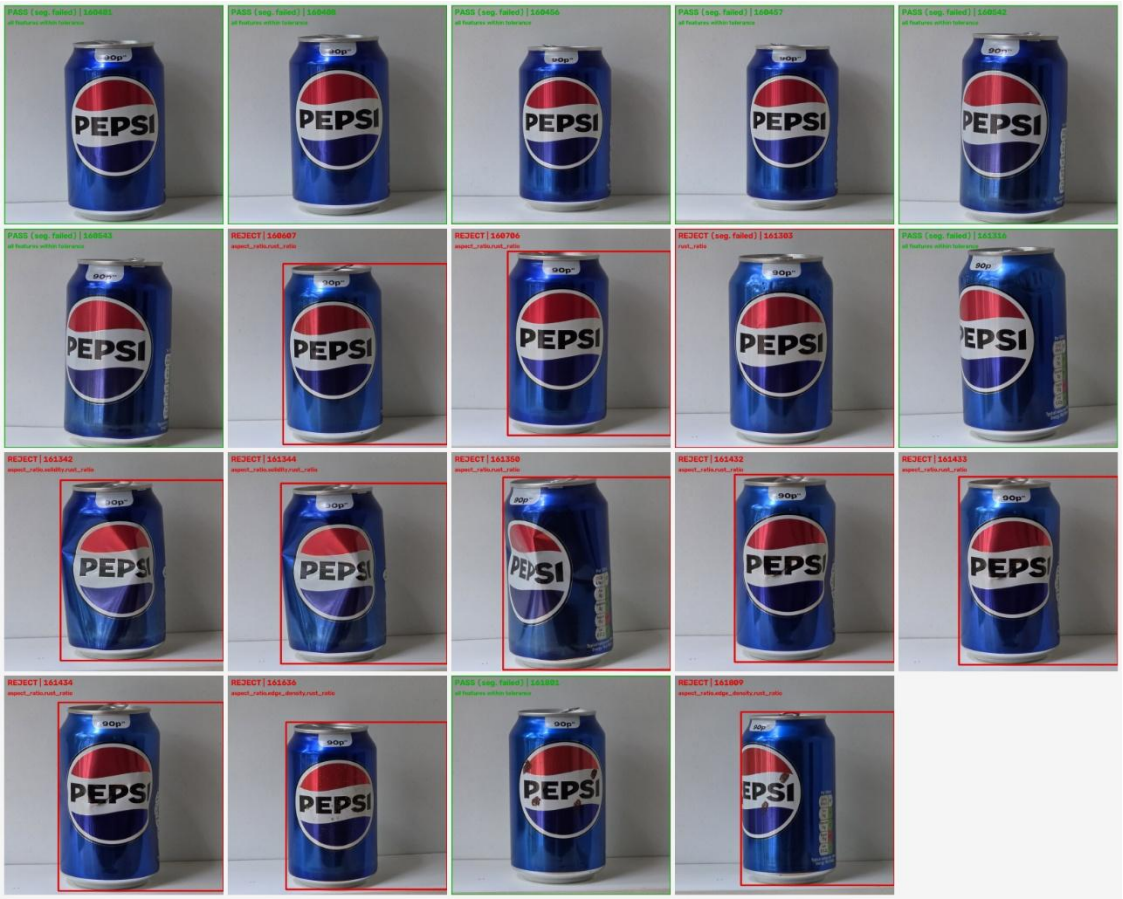


Figure A.2: Annotated output for all nineteen real photographs in Table A.2, generated by re-running the pipeline's own segment_can and feature-extraction functions against each photograph and drawing its bounding box and vision-only decision (real_image_pipeline.py). Green border/text = PASS; red border/text = REJECT with the anomalous feature(s) printed beneath the decision; a full-frame border indicates the segmentation-failure mode discussed in section 4.8.

Appendix B: Key code excerpts

The complete, executable pipeline is provided as the Colab notebook Real_Tin_Can_Inspection_Colab.ipynb and as the standalone module coppeliasim_tin_can_project/inspection.py; both are submitted alongside this report. The two excerpts below show the robust anomaly-scoring step and the decision-fusion step referenced in sections 3.5 and 3.8.

B.1 Robust (median/MAD) anomaly scoring

```
robust_z = ((pd.Series({k: m[k] for k in FEATURE_COLUMNS}) - baseline_median).abs()
            / (1.4826 * baseline_mad)).replace([np.inf], np.nan).fillna(0)
anomaly_score = float(robust_z.max())
anomalous_features = ','.join(robust_z[robust_z > ANOMALY_THRESHOLD].index) or 'none'
```

B.2 Decision fusion

```
visual_ok = not detected_defects if yolo_model is not None else anomaly_score <=
ANOMALY_THRESHOLD
weight_ok = abs(weight_g - NOMINAL_WEIGHT_G) <= WEIGHT_TOLERANCE_G
if not visual_ok: reasons.append('visual:' + (','.join(detected_defects) or
anomalous_features))
if not weight_ok: reasons.append('weight')
if OCR_AFFECTS_DECISION and not ocr_ok: reasons.append('ocr_unreadable')
```

```
if OCR_AFFECTS_DECISION and not expiry_ok: reasons.append('expiry_missing_or_expired')
decision = 'PASS' if not reasons else 'REJECT'
```

Appendix C: CoppeliaSim project structure

The CoppeliaSim project (coppeliasim_tin_can_project/) submitted alongside this report contains:

- build_scene.py: builds the conveyor, gantry, sensors, pusher, chutes and bins described in section 3.9;
- run_simulation.py: the synchronised-stepping controller that calls the real inspection pipeline per can and drives the sorting motion;
- inspection.py: the standalone inspection module shared with the Colab notebook;
- real_image_pipeline.py: added for this report; re-runs inspection.py's own segment_can and feature-extraction functions, unmodified, against the real photographs described in section 3.12 and reproduces the section 4.8 vision-channel results;
- config.yaml: simulation and inspection configuration (tolerances, thresholds, file paths);
- data/: the evidence set described in section 3.3 (input_images/, good_reference/, weights.csv);
- output/: results CSV (inspection_results.csv), the real-image results CSV (real_inspection_results.csv), annotated images and virtual-camera captures generated on each run.

The real-image evidence set described in section 3.12 is the nineteen photographs in Datasets/Colab_Training_Data/Good_Cans/ (eight clean references) and Datasets/Colab_Training_Data/Damaged_Cans/ (eleven staged-defect photographs); real_image_pipeline.py reads directly from these folders and writes real_inspection_results.csv, the full results table reproduced in Appendix A.2. Annotated bounding-box images for the real photographs were not regenerated as part of this reproduction pass.

Appendix D: Use of generative AI

In line with the assessment brief, generative AI tools were used during the preparation of this report to support learning and understanding of the underlying methods and literature, to assist with structuring, cross-checking and formatting the write-up against the Department of Engineering report template, and to create the flowchart diagrams used throughout (Figures 3.2- 3.5, 4.8 and 6.1). The project design, implementation, evidence, results and analysis are the author's own work.